IFA Preload DD Kernel 完整代码逐行解析

源文件: incre_flash_attention_preload_dd.h (3991行)

说明: 本文档包含所有核心代码，智能换行处理，语法高亮显示

一、头部宏定义与常量 (行1-40)

功能说明: 包含PRE_LOAD_NUM_DD、EXTRA_NUM、TASK_NUM等关键参数定义

```
/**
 * Copyright (c) 2025 Huawei Technologies Co., Ltd.
 * This program is free software, you can redistribute it and/or modify
    // ↑ 换行续接 ↓
     it under the terms and conditions of
 * CANN Open Software License Agreement Version 2.0 (the "License").
 * Please refer to the License for details. You may not use this file except
    // ↑ 换行续接 ↓
     in compliance with the License.
 * THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, WITHOUT WARRANTIES OF ANY KIND,
    // ↑ 换行续接 ↓
     EITHER EXPRESS OR IMPLIED,
 * INCLUDING BUT NOT LIMITED TO NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS
    // ↑ 换行续接 ↓
     FOR A PARTICULAR PURPOSE.
 * See LICENSE in the root of the software repository for the full text of the License.
 */

/*!
 * \file incre_flash_attention_preload_dd.h
 * \brief
 */
#ifndef INCRE_FLASH_ATTENTION_PRELOAD_DD
#define INCRE_FLASH_ATTENTION_PRELOAD_DD

#if ASC_DEVKIT_MAJOR >= 9
#include "kernel_vec_intf.h"
#include "kernel_cube_intf.h"
#else
#include "kernel_operator.h"
#endif
#include "kernel_operator_list_tensor_intf.h"
#include "kernel_tiling/kernel_tiling.h"
#include "lib/matmul_intf.h"
#include "lib/matrix/matmul/tiling.h"
#include "../ifa_public_define.h"

using namespace matmul;
using AscendC::CacheMode;
using AscendC::CrossCoreSetFlag;
using AscendC::CrossCoreWaitFlag;

#define PRE_LOAD_NUM_DD 4
#define ATTENMASK_STRIDE 2048U
#define EXTRA_NUM 6
#define TASK_NUM 2
#define IFA_SOFTMAX_WITHOUT_BRC_PRELOAD
#define Q_AMAX_BUF_BYTES (BUFFER_SIZE_BYTE_2K + BUFFER_SIZE_BYTE_256B)
```

二、核心参数结构体定义 (行41-100)

功能说明: Tiling参数和Buffer参数结构体

```
#define P_AMAX_BUF_BYTES (BUFFER_SIZE_BYTE_2K + BUFFER_SIZE_BYTE_256B)
#define Q_ROWSUM_BUF_BYTES (BUFFER_SIZE_BYTE_512B + BUFFER_SIZE_BYTE_256B)
#define P_ROWSUM_BUF_BYTES (BUFFER_SIZE_BYTE_512B + BUFFER_SIZE_BYTE_256B)
#define Q_AMAX_BUF_SIZE (Q_AMAX_BUF_BYTES / sizeof(T))
#define P_AMAX_BUF_SIZE (P_AMAX_BUF_BYTES / sizeof(T))
#define Q_ROWSUM_BUF_SIZE (Q_ROWSUM_BUF_BYTES / sizeof(T))
```

```
#define P_ROWSUM_BUF_SIZE (P_ROWSUM_BUF_BYTES / sizeof(T))

#define SOFTMAX_MAX_BUF_BYTES BUFFER_SIZE_BYTE_512B
#define SOFTMAX_EXP_BUF_BYTES BUFFER_SIZE_BYTE_512B
#define SOFTMAX_SUM_BUF_BYTES BUFFER_SIZE_BYTE_512B
#define SOFTMAX_MAX_BUF_SIZE (SOFTMAX_MAX_BUF_BYTES / sizeof(T))
#define SOFTMAX_EXP_BUF_SIZE (SOFTMAX_EXP_BUF_BYTES / sizeof(T))
#define SOFTMAX_SUM_BUF_SIZE (SOFTMAX_SUM_BUF_BYTES / sizeof(T))

template <typename IFAT> class IncreFlashAttentionAttenPreloadDD {
struct ExtraInfo {
    uint32_t loop = 0;
    uint32_t bIdx = 0;
    uint32_t gIdx = 0;
    uint32_t s1Idx = 0;
    uint32_t n2Idx = 0;
    uint32_t s2Idx = 0;
    uint32_t bn2IdxInCurCore = 0;
    uint32_t curSInnerLoopTimes = 0;
    uint64_t tensorAOffset = 0;
    uint64_t tensorBOffset = 0;
    uint64_t attenOutOffset = 0;
    uint64_t attenMaskOffset = 0;
    uint64_t antiqKeyParamOffset = 0;
    uint64_t antiqValueParamOffset = 0;
    uint64_t antiqKeyParamOffsetPerToken = 0;
    uint64_t antiqValueParamOffsetPerToken = 0;
    uint64_t perChannelQuantOffset = 0ULL;
    uint32_t actualSingleProcessSInnerSize = 0;
    uint32_t actualSingleProcessSInnerSizeAlign = 0;
    uint32_t s2BatchOffset = 0;
    uint32_t s1Size = 0;
    uint32_t actS1Size = 0;
    uint32_t s2Size = 0;
    uint32_t gSize = 0;
    uint32_t mSize = 0;
    uint32_t mSizeV = 0;
    uint32_t mSizeVStart = 0;
    uint64_t actSeqLenQ = 0;
    bool isFirstSInnerLoop = false;
    bool isChangeBatch = false;
};

struct TaskContext {
    uint32_t bidx;
    uint32_t gidx;
    uint32_t s1idx;
    uint32_t s2idx;
    uint32_t s2loops;
    uint32_t s2SizeTail;
    uint32_t s1Size;
    uint32_t s2Size;
    uint32_t isFirstLoop;
    uint32_t nidx;
```

三、类定义与成员变量 (行101-550)

功能说明: 包含所有成员变量、Buffer声明、函数声明

```
    uint64_t actS1Size = 1;
    uint64_t actSeqLenQ = 0;
};

public:
    __aicore__ inline IncreFlashAttentionAttenPreloadDD(){};
    __aicore__ inline void Init(__gm__ uint8_t *query, __gm__ uint8_t *key,
        // ↑ 换行续接 ↓
        __gm__ uint8_t *value,
                                __gm__ uint8_t *pseShift, __gm__ uint8_t *attenMask,
                                    // ↑ 换行续接 ↓
                                    __gm__ uint8_t *actualSeqLengthsQ,
                                __gm__ uint8_t *actualSeqLengths, __gm__
                                    // ↑ 换行续接 ↓
```

```
                                uint8_t *blockTable,
                            __gm__ uint8_t *kvPaddingSize, __gm__ uint8_t *attentionOut,
                                // ↑ 换行续接 ↓
                                __gm__ uint8_t *softmaxLse,
                            __gm__ uint8_t *workspace, const IncreFlashAttentionTilingData
                                // ↑ 换行续接 ↓
                                *__restrict tiling,
                            __gm__ uint8_t *gmTiling, TPipe *tPipe,
                                // ↑ 换行续接 ↓
                                bool isPrefix = false);
__aicore__ inline void InitQuant(__gm__ uint8_t *deqScale1, __gm__ uint8_t *quantScale1,
    // ↑ 换行续接 ↓
    __gm__ uint8_t *deqScale2,
                                 __gm__ uint8_t *quantScale2, __gm__
                                    // ↑ 换行续接 ↓
                                    uint8_t *quantOffset2,
                                 __gm__ uint8_t *antiquantScale, __gm__
                                    // ↑ 换行续接 ↓
                                    uint8_t *antiquantOffset,
                                 __gm__ uint8_t *keyAntiquantScale,
                                    // ↑ 换行续接 ↓
                                    __gm__ uint8_t *keyAntiquantOffset,
                                 __gm__ uint8_t *valueAntiquantScale,
                                    // ↑ 换行续接 ↓
                                    __gm__ uint8_t *valueAntiquantOffset,
                                 __gm__ uint8_t *workspace);
__aicore__ inline void InitAntiquant(__gm__ uint8_t *antiquantScale,
    // ↑ 换行续接 ↓
    __gm__ uint8_t *antiquantOffset,
                                   __gm__ uint8_t *keyAntiquantScale,
                                      // ↑ 换行续接 ↓
                                      __gm__ uint8_t *keyAntiquantOffset,
                                   __gm__ uint8_t *valueAntiquantScale,
                                      // ↑ 换行续接 ↓
                                      __gm__ uint8_t *valueAntiquantOffse);
__aicore__ inline void InitPostQuant(__gm__ uint8_t *deqScale1, __gm__ uint8_t *quantScale1,
    // ↑ 换行续接 ↓
    __gm__ uint8_t *deqScale2,
                                   __gm__ uint8_t *quantScale2, __gm__
                                      // ↑ 换行续接 ↓
                                      uint8_t *quantOffset2);
__aicore__ inline void Process();

// 中间计算数据类型为float，高精度模式
using T = float;

__aicore__ inline void Bmm2ResCopyOut(const ExtraInfo &info, LocalTensor<T>
    // ↑ 换行续接 ↓
    &bmm2ResUb, uint32_t startRow,
                                  uint32_t dealRowCount, uint32_t columnCount,
                                     // ↑ 换行续接 ↓
                                     uint32_t actualColumnCount);
using Q_T = typename IFAT::queryType;
using KV_T = typename IFAT::kvType;
using OUT_T = typename IFAT::outputType;
using ORIGIN_T = typename IFAT::orginalType;
static constexpr bool PAGE_ATTENTION = IFAT::pageAttention;
static constexpr bool KV_CONTINUOUS = IFAT::kvContinuous;
static constexpr bool FLASH_DECODE = IFAT::flashDecode;
static constexpr LAYOUT LAYOUT_T = IFAT::layout;
static constexpr LAYOUT KV_LAYOUT_T = IFAT::kvLayout;
static constexpr uint8_t PER_CHANNEL_MODE = 0; // 伪量化: K V per-channel
static constexpr uint8_t PER_TOKEN_MODE = 1; // 伪量化: K V per-token
static constexpr uint8_t PER_CHANNEL_TOKEN_MODE = 2; // 伪量化: K per-channel
    // ↑ 换行续接 ↓
    and V per-token
static constexpr uint8_t ANTIQUANT_MODE = IFAT::antiquantMode;

static constexpr bool ANTIQUANT = !IsSameType<Q_T, KV_T>::value;
static constexpr bool KVINT4 = IsSameType<KV_T, int4b_t>::value;
static constexpr bool QUANT = (IsSameType<Q_T, KV_T>::value && IsSameType<KV_T,
    // ↑ 换行续接 ↓
    int8_t>::value);
static constexpr bool ANTIQUANT_PER_CHANNEL_TOKEN = (ANTIQUANT && (ANTIQUANT_MODE
    // ↑ 换行续接 ↓
    == PER_CHANNEL_TOKEN_MODE));
static constexpr bool ANTIQUANT_PER_TOKEN = (ANTIQUANT && (ANTIQUANT_MODE
    // ↑ 换行续接 ↓
    == PER_TOKEN_MODE));
```

```
    static constexpr bool ANTIQUANT_PER_CHANNEL = (ANTIQUANT && (ANTIQUANT_MODE
        // ↑ 换行续接 ↓
        == PER_CHANNEL_MODE));
    using ANTIQ_PARAMS_T = typename AscendC::Conditional<ANTIQUANT_PER_TOKEN,
        // ↑ 换行续接 ↓
        T, Q_T>::type;

    static constexpr bool POST_QUANT = IsSameType<OUT_T, int8_t>::value;
    using MM_OUT_T = typename AscendC::Conditional<ANTIQUANT, half, T>::type;
    using L0C_T = typename AscendC::Conditional<ANTIQUANT, int32_t, T>::type;

protected:
    const IncreFlashAttentionTilingData *__restrict tilingData = nullptr;
    TPipe *pipe = nullptr;

    GlobalTensor<Q_T> queryGm;
    GlobalTensor<KV_T> keyGm;
    GlobalTensor<KV_T> valueGm;
    GlobalTensor<OUT_T> attentionOutGm;
    GlobalTensor<float> softmaxLseGm;
    GlobalTensor<int32_t> blockTableGm;

    // atten mask
    GlobalTensor<bool> attenMaskBoolGm;

    GlobalTensor<ANTIQ_PARAMS_T> keyAntiqOffsetGm;
    GlobalTensor<ANTIQ_PARAMS_T> keyAntiqScaleGm;
    GlobalTensor<ANTIQ_PARAMS_T> valueAntiqOffsetGm;
    GlobalTensor<ANTIQ_PARAMS_T> valueAntiqScaleGm;
    GlobalTensor<uint64_t> actualSeqLengthsGm;
    GlobalTensor<uint64_t> actualSeqLengthsGmQ;
    // out quant
    GlobalTensor<float> quantScale2Gm;
    GlobalTensor<float> quantOffset2Gm;
    GlobalTensor<bfloat16_t> quantScale2Bf16Gm;
    GlobalTensor<bfloat16_t> quantOffset2Bf16Gm;
    // workspace
    GlobalTensor<KV_T> queryPreProcessResGm;
    GlobalTensor<MM_OUT_T> mm1ResGm;
    GlobalTensor<KV_T> vec1ResGm;
    GlobalTensor<MM_OUT_T> mm2ResGm;
    GlobalTensor<MM_OUT_T> vec2ResGm;

    GlobalTensor<T> accumOutGm;
    GlobalTensor<T> lseSumFdGm;
    GlobalTensor<T> lseMaxFdGm;

    // kv_left_padding
    GlobalTensor<int64_t> kvPaddingSizeGm;

    // queue
    TBuf<> inputBuf1; // 32K, inque
    TBuf<> inputBuf2; // 16K, inque
    TBuf<> outputBuf1; // 32K, outque
    TBuf<> outputBuf2; // 8K, outque
    static constexpr uint64_t SYNC_INPUT_BUF1_FLAG = 2;
    static constexpr uint64_t SYNC_INPUT_BUF2_FLAG = 3;
    static constexpr uint64_t SYNC_OUTPUT_BUF1_FLAG = 4;
    static constexpr uint64_t SYNC_OUTPUT_BUF2_FLAG = 5;

    TBuf<TPosition::A1> tmpBufL1Q;
    LocalTensor<KV_T> qL1Buffers;
    TBuf<TPosition::A1> tmpBufL1KP;
    LocalTensor<KV_T> kpL1Buffers;
    TBuf<TPosition::A1> tmpBufL1V;
    LocalTensor<KV_T> vL1Buffers;

    TBuf<TPosition::A2> tmpBufL0A;
    LocalTensor<KV_T> aL0TensorPingPong;
    // L0_B
    TBuf<TPosition::B2> tmpBufL0B;
    LocalTensor<KV_T> bL0TensorPingPong;
    // L0_C
    TBuf<TPosition::CO1> tmpBufL0C;
    LocalTensor<L0C_T> cL0TensorPingPong;

    // 临时tbuf
    TBuf<> tmpBuff1; // 32K
```

```
    TBuf<> tmpBuff2; // 32K
    TBuf<> tmpBuff3; // 2K

    // 常驻tbuf
    TBuf<> vec2ResBuff; // 16k 伪量化场景
    TBuf<> antiqScaleBuff; // 4K
    TBuf<> antiqOffsetBuff; // 4K
    TBuf<> qAmaxBuff; // PRE_LOAD_NUM_DD * (2K + 256B)
    TBuf<> softmaxResAmaxBuff; // 2K + 256B
    TBuf<> qRowSumBuff; // 2K + 256B
    TBuf<> softmaxResRowSumBuff; // 2K + 256B
    TBuf<> softmaxMaxBuff; // PRE_LOAD_NUM_DD * 2K
    TBuf<> softmaxExpBuff; // PRE_LOAD_NUM_DD * 2K
    TBuf<> softmaxSumBuff; // PRE_LOAD_NUM_DD * 2K
    TBuf<> softmaxMaxDefaultBuff; // 2K
    TBuf<> softmaxSumDefaultBuff; // 2K

    LocalTensor<T> softmaxMaxUb;
    LocalTensor<T> softmaxSumUb;
    LocalTensor<T> softmaxExpUb;
    LocalTensor<T> softmaxMaxDefaultUb;
    LocalTensor<T> softmaxSumDefaultUb;

    // antiquant msd
    LocalTensor<T> aMaxBmm1Ub;
    LocalTensor<T> aMaxBmm2Ub;
    LocalTensor<T> softmaxScaleResRowSumUb;
    LocalTensor<half> vec2ResUb;
    LocalTensor<T> antiqScaleUb;
    LocalTensor<T> antiqOffsetUb;
    LocalTensor<T> qRowSumUb;

    static constexpr float quantScaleC1S1 = 1.0 / (1024);
    static constexpr float quantScaleC1S2 = 1.0 / (1024 * 254);
    static constexpr float quantScaleC2O1 = 1.0 / (1024 * 127);
    static constexpr float quantScaleC2O2 = 1.0 / (1024 * 254 * 127);

    // L1
    static constexpr uint32_t L1Q_BLOCK_SIZE = 64 * 1024; // 64k
    static constexpr uint32_t L1KP_BLOCK_SIZE = 64 * 1024; // 64k
    static constexpr uint32_t L1V_BLOCK_SIZE = 64 * 1024; // 64k

    // L0
    static constexpr uint32_t L0A_PP_SIZE = (32 * 1024);
    static constexpr uint32_t L0B_PP_SIZE = (32 * 1024);
    static constexpr uint32_t L0C_PP_SIZE = (64 * 1024);

    // mte2 <> mte1
    static constexpr uint32_t L1Q_EVENT0 = EVENT_ID0;
    static constexpr uint32_t L1KP_EVENT0 = EVENT_ID1;
    static constexpr uint32_t L1KP_EVENT1 = EVENT_ID2;
    static constexpr uint32_t L1KP_EVENT2 = EVENT_ID3;
    static constexpr uint32_t L1V_EVENT0 = EVENT_ID4;
    static constexpr uint32_t L1V_EVENT1 = EVENT_ID5;
    static constexpr uint32_t L1V_EVENT2 = EVENT_ID6;
    static constexpr uint32_t L1V_EVENT3 = EVENT_ID7;

    // m <> mte1
    static constexpr uint32_t L0A_EVENT0 = EVENT_ID3;
    static constexpr uint32_t L0A_EVENT1 = EVENT_ID4;
    static constexpr uint32_t L0B_EVENT0 = EVENT_ID5;
    static constexpr uint32_t L0B_EVENT1 = EVENT_ID6;

    // fix <> m
    static constexpr uint32_t L0C_EVENT0 = EVENT_ID3;
    static constexpr uint32_t L0C_EVENT1 = EVENT_ID4;

    uint32_t qL1BufIter = 0;
    uint32_t kpL1BufIter = -1;
    uint32_t vL1BufIter = -1;
    uint32_t aL0BufIter = 0;
    uint32_t bL0BufIter = 0;
    uint32_t cL0BufIter = 0;

    static constexpr uint64_t SYNC_MODE2 = 2;
    static constexpr uint64_t SYNC_V0_C1_FLAG = 6;
    static constexpr uint64_t SYNC_C1_V1_FLAG = 7;
    static constexpr uint64_t SYNC_V1_C2_FLAG = 8;
```

```
    static constexpr uint64_t SYNC_C2_V2_FLAG = 9;

    static constexpr uint32_t BLOCK_ELEMENT_NUM = BYTE_BLOCK / sizeof(T);
    static constexpr uint32_t REPEAT_ELEMENT_NUM = REPEAT_BLOCK_BYTE / sizeof(T);
    static constexpr uint32_t BASE_BLOCK_MAX_ELEMENT_NUM = BUFFER_SIZE_BYTE_32K
        // ↑ 换行续接 ↓
        / sizeof(T);
    static constexpr uint32_t ADDRESS_ALIGN_NUM = 512 / sizeof(KV_T);
    static constexpr uint32_t ADDRESS_ALIGN_NUM_THRESHLOD = 128 / sizeof(KV_T);
    MM_OUT_T antiquantExpandCoeff = KVINT4 ? 14.98 : 254;
    static constexpr T antiqCoeff1 = KVINT4 ? 7.49 : 127;
    static constexpr T antiqCoeff2 = 1 / antiqCoeff1;
    static constexpr T SOFTMAX_MIN_NUM = -2e38;
    static constexpr T BOOL_ATTEN_MASK_SCALAR_VALUE = -1000000000000.0; // 用于mask为bool类型
    static constexpr T FP16_ATTEN_MASK_SCALAR_VALUE = -10000;          // 用于mask为fp16类型
    static constexpr uint32_t P_LOAD_TO_L1_ROW_NUM = 128 / sizeof(KV_T);
    static constexpr uint32_t KV_LOAD_TO_L1_ROW_NUM = 512 / sizeof(KV_T);
    bool antiqOffsetExistFlag = false;
    uint32_t msdIterNum = 0U;
    bool antiquantPerHeadFlag = false;
    bool antiquantParamsInPagedAttentionFlag = false;
    uint32_t antiquantPerTensorFlag = 0U;

    // kv_left_padding
    uint32_t kvPaddingFlag = 0;
    uint64_t kvPaddingBeginOffset = 0;

    // for workspace pingpong
    const uint32_t dbWorkspaceRatio = PRE_LOAD_NUM_DD;

    __gm__ uint8_t *keyPtr = nullptr;
    __gm__ uint8_t *valuePtr = nullptr;

    __gm__ uint8_t *key_ = nullptr;
    __gm__ uint8_t *value_ = nullptr;

    uint32_t tmpBlockIdx = 0U;
    uint32_t aiCoreIdx = 0U;
    __gm__ uint8_t *blocktablePtr = nullptr;

    // tilingdata
    uint64_t singleProcessSInnerSize = 0U;
    uint32_t sInnerLoopTimes = 0U;
    uint64_t singleProcessSInnerSizeTail = 0U;
    uint32_t formerCoreNum = 0U;
    uint32_t usedCoreNum = 0U;
    uint32_t bIdx = 0U;
    uint32_t n2Idx = 0U;

    uint32_t vec1ResUbSize = 0U;
    uint32_t mmResUbSize = 0U;
    uint32_t bmm2ResUbSize = 0U;
    uint32_t batchContinuous = 0U;

    uint64_t batchSize = 0ULL;
    uint64_t qHeadNum = 0ULL;
    uint64_t kvHeadNum = 0ULL;
    uint64_t gSize = 0ULL;

    uint64_t actS1Size = 1ULL;
    uint64_t gSizeSub = 0ULL;
    uint64_t gSizeTail = 0ULL;
    uint64_t s1SizeSub = 0ULL;
    uint64_t s1SizeTail = 0ULL;
    uint64_t gOuter = 1ULL;
    uint64_t s1Outer = 1ULL;
    uint64_t gIdx = 0ULL;
    uint32_t s1Idx = 0U;
    uint64_t mSizeVector = 0ULL;
    uint64_t mSizeVStart = 0ULL;
    uint64_t qSeqSize = 0ULL;

    uint64_t kvSeqSize = 0ULL;
    uint64_t headDim = 0ULL;
    uint64_t headDimAlign = 0ULL;

    // pageAttention
    uint32_t kvCacheBlockSize = 0;
```

```
    uint32_t maxBlockNumPerBatch = 0;
    uint64_t s2BatchBaseOffset = 0;

    // 是否返回lse
    bool softmaxLseFlag = false;

    // attention mask
    bool attenMaskFlag = false;
    uint32_t selectWithByteMaskTmpMinSize = 0U;
    uint32_t attenMaskSizeAlign = 0U;
    // pse mask
    uint32_t pseMaskSizeAlign = 0U;
    // offset
    uint64_t tensorACoreOffset = 0ULL;
    uint64_t tensorBCoreOffset = 0ULL;
    uint64_t tensorBOffset = 0ULL;
    uint64_t valueOffset = 0ULL;
    uint64_t attenOutOffset = 0ULL;
    uint64_t antiqParamOffset = 0ULL;
    uint64_t attenMaskOffset = 0ULL;
    uint64_t attenMaskCoreOffset = 0ULL;
    uint64_t antiqKeyParamCoreOffsetPerToken = 0ULL;
    uint64_t antiqParamOffsetPerToken = 0ULL;
    uint64_t attenMaskSize = 0ULL;
    uint64_t antiqSeqSize = 0ULL;

    // splitKV
    uint32_t splitKVNum = 0U;
    uint32_t s2Idx = 0U;
    uint32_t s2IdxFD = 0U;
    uint64_t sInnerLoopSize = 0ULL;
    uint32_t actualCombineLoopSize = 0U;
    uint64_t combineLseOffset = 0ULL;
    uint64_t combineAccumOutOffset = 0ULL;

    uint64_t curActualSeqLen = 0ULL;
    uint64_t curActualSeqLenQ = 0ULL;
    uint32_t beforeBlockSplitBn2Nums = 0U;
    uint32_t bn2LoopTimes = 0U;

    uint32_t actualLenDims = 0U;
    uint32_t actualLenQDims = 0U;
    // out quant
    bool isPerChnU8Out = false;
    bool isOutQuantTypeBf16 = false;
    float quantScale2Value = 0;
    float quantOffset2Value = 0;
    bool isQuantOffset2Exist = false;
    uint64_t perChannelQuantOffset = 0ULL;
    static constexpr float scaleC1 = 1024.0 / 127;
    static constexpr float scaleC2 = 1024.0;

    // 记录当前轮的bIdx nIdx s2Idx actualLen
    uint32_t bn2IdxInCurCore = 0;
    uint32_t bn2s2LoopTimes = 0;
    __aicore__ inline void InitValueGm(uint32_t bIdx);
    __aicore__ inline void InitKeyGm(uint32_t bIdx);
    __aicore__ inline void CalcParams(uint32_t loop, ExtraInfo& info, TaskContext &task);
    __aicore__ inline void ComputeMm1(ExtraInfo& info);
    __aicore__ inline void ProcessVec1L(ExtraInfo& info);
    __aicore__ inline void ComputeMm2(ExtraInfo& info);
    __aicore__ inline void ProcessVec2L(ExtraInfo& info);
    __aicore__ inline void SetLoopTimes();
    __aicore__ inline void QueryPreProcessL(ExtraInfo& info);
    __aicore__ inline void CopyInMm1AToL1(LocalTensor<KV_T>& aL1Tensor, ExtraInfo& info);
    __aicore__ inline void CopyInMm1BToL1(LocalTensor<KV_T>& bL1Tensor, ExtraInfo& info,
        // ↑ 换行续接 ↓
        uint32_t nCopyIdx, uint32_t nCopyRowCount, uint32_t nActCopyRowCount, uint32_t
nActCopyRowCountAlign);
    __aicore__ inline void CopyInMm1BToL1ForPA(LocalTensor<KV_T>& bL1Tensor,
        // ↑ 换行续接 ↓
        uint64_t keyGmBaseOffset, uint32_t copyTotalRowCnt, uint32_t copyStartRowCnt, uint32_t nActCopyRowCo
unt);
    __aicore__ inline void LoadDataMm1A(ExtraInfo& info, LocalTensor<KV_T>& aL0Tensor,
        // ↑ 换行续接 ↓
        LocalTensor<KV_T>& aL1Tensor);
    __aicore__ inline void LoadDataMm1B(LocalTensor<KV_T>& bL0Tensor, LocalTensor<KV_T>&
        // ↑ 换行续接 ↓
```

```
        bL1Tensor);

    __aicore__ inline void CopyInMm2AToL1(LocalTensor<KV_T>& aL1Tensor, ExtraInfo& info,
        // ↑ 换行续接 ↓
        uint32_t mCopyIdx, uint32_t mCopyRowCount, uint32_t mActCopyRowCount, uint32_t kCopyIdx, uint32_t kC
opyRowCount, uint32_t kActCopyRowCount);
    __aicore__ inline void CopyInMm2BToL1(LocalTensor<KV_T>& bL1Tensor, ExtraInfo& info,
        // ↑ 换行续接 ↓
        uint32_t kCopyIdx, uint32_t kCopyRowCount, uint32_t kActCopyRowCount);
    __aicore__ inline void CopyInMm2BToL1ForPA(LocalTensor<KV_T>& bL1Tensor,
        // ↑ 换行续接 ↓
        uint64_t valueGmBaseOffset, uint32_t copyTotalRowCnt, uint32_t copyStartRowCnt, uint32_t kActCopyRow
Count);
    __aicore__ inline void LoadDataMm2A(LocalTensor<KV_T> aL0Tensor, LocalTensor<KV_T>
        // ↑ 换行续接 ↓
        aL1Tensor, uint32_t kSize);
    __aicore__ inline void LoadDataMm2B(LocalTensor<KV_T>& bL0Tensor, LocalTensor<KV_T>&
        // ↑ 换行续接 ↓
        bL1Tensor);

    bool curActSeqLenIsZero = false;

    template <typename T> __aicore__ inline T Align(T num, T rnd)
    {
        return (((rnd) == 0) ? 0 : (((num) + (rnd) - 1) / (rnd) * (rnd)));
    }
    __aicore__ inline void InitTilingData();
    __aicore__ inline void InitCalcParams();
    __aicore__ inline void InitCalcParamsEach();
    __aicore__ inline void InitBuffers();
    __aicore__ inline void InitActualSeqLen(__gm__ uint8_t *actualSeqLengthsQ,
        // ↑ 换行续接 ↓
        __gm__ uint8_t *actualSeqLengths);
    __aicore__ inline uint64_t GetQueryOffSetBySIdx(uint32_t bIdx, uint32_t sIdx,
        // ↑ 换行续接 ↓
        uint32_t n2Idx, uint32_t gIdx);
    __aicore__ inline void GetTndActualSeqLenQ(uint32_t bIdx, uint32_t s1Idx);
    __aicore__ inline void GetActualSeqLen(uint32_t bIdx, uint32_t s1Idx = 0);
    __aicore__ inline void UpdateInnerLoopCond();
    __aicore__ inline void DealActSeqLenIsZero(uint32_t bIdx, uint32_t n2Idx);
    __aicore__ inline bool ComputeKVPaddingBeginOffset();

    __aicore__ inline void GetBN2Gid(const uint32_t bn2Idx);

    __aicore__ inline void CalcSInnerOffsetAndParams(const uint32_t sInnerLoopIdx);

    __aicore__ inline void AttenMaskCopyIn(uint64_t offset, uint32_t dealRowCount,
        // ↑ 换行续接 ↓
        uint32_t actualColumnCount);
    __aicore__ inline void AttenMaskCopyIn(const ExtraInfo& info);

    __aicore__ inline void CopyAntiquantScale(LocalTensor<T> &castUb, GlobalTensor<Q_T>
        // ↑ 换行续接 ↓
        srcGm, uint64_t offset);

    __aicore__ inline void CopyAntiquantParamsPerToken(GlobalTensor<ANTIQ_PARAMS_T>
        // ↑ 换行续接 ↓
        srcGm, uint64_t offset,
                                                       uint32_t columnCount,
                                                           // ↑ 换行续接 ↓
                                                           uint32_t actualColumnCount);
    __aicore__ inline void CopyAntiquantParamsPerTokenHead(GlobalTensor<ANTIQ_PARAMS_T>
        // ↑ 换行续接 ↓
        srcGm, uint64_t offset,
                                                           uint32_t columnCount);
    __aicore__ inline void CopyAntiquantParamsPerTokenParamsPagedAttention(
        // ↑ 换行续接 ↓
        GlobalTensor<ANTIQ_PARAMS_T> srcGm,
                                                                  uint64_t offset,
                                                                     // ↑ 换行续接 ↓
                                                                     uint32_t actualColumnCount);
    __aicore__ inline void CopyAntiqQuery(LocalTensor<T> &queryCastUb, uint64_t qOffset,
        // ↑ 换行续接 ↓
        uint32_t dealRowCount,
                                          uint32_t columnCount, uint32_t
                                             // ↑ 换行续接 ↓
                                             actualColumnCount);
    __aicore__ inline void AbsRowMax(LocalTensor<T> &tmpAMaxRes, LocalTensor<T> &srcUb,
        // ↑ 换行续接 ↓
```

```
    LocalTensor<T> tmpAUb,
                                uint32_t dealRowCount, uint32_t columnCount,
                                    // ↑ 换行续接 ↓
                                uint32_t actualColumnCount);
__aicore__ inline void AntiquantAIterExpand(GlobalTensor<KV_T> dstGm,
    // ↑ 换行续接 ↓
    LocalTensor<half> &tmpA1, LocalTensor<half> &tmpA2,
                                        uint32_t calcSize, bool isFirst,
                                           // ↑ 换行续接 ↓
                                        uint64_t outOffset);
__aicore__ inline void AntiquantMatmulPreProcess(ExtraInfo& info, GlobalTensor<KV_T> dstGm,
    // ↑ 换行续接 ↓
    LocalTensor<T> aMaxResUb,
                                           LocalTensor<T> srcUb,
                                              // ↑ 换行续接 ↓
                                              LocalTensor<T> tmpAFloorUb,
                                           uint32_t startRow, uint32_t dealRowCount,
                                              // ↑ 换行续接 ↓
                                              uint32_t columnCount,
                                           uint32_t actualColumnCount);
__aicore__ inline void AntiquantSoftmaxResPreProcess(ExtraInfo& info,
    // ↑ 换行续接 ↓
    GlobalTensor<KV_T> dstGm, LocalTensor<T> srcUb,
                                              LocalTensor<T>
                                                 // ↑ 换行续接 ↓
                                                 tmpAFloorUb, uint32_t startRow,
                                              uint32_t dealRowCount,
                                                 // ↑ 换行续接 ↓
                                                 uint32_t columnCount,
                                              uint32_t actualColumnCount);
__aicore__ inline void DealQueryPreProcessBaseBlock(ExtraInfo& info,
    // ↑ 换行续接 ↓
    uint32_t startRow, uint32_t dealRowCount, uint32_t columnCount,
                                             uint32_t actualColumnCount);
__aicore__ inline void DealQueryPreProcessBaseBlockPerToken(ExtraInfo& info,
    // ↑ 换行续接 ↓
    uint32_t startRow, uint32_t dealRowCount,
                                                    uint32_t columnCount,
                                                       // ↑ 换行续接 ↓
                                                       uint32_t actualColumnCount);
__aicore__ inline void QueryPreProcessInner(ExtraInfo& info);

__aicore__ inline void FlashDecodeCompute();
__aicore__ inline void Bmm1ComputeCommon(const uint32_t bn2Idx, const
    // ↑ 换行续接 ↓
    uint32_t sInnerLoopIdx);
__aicore__ inline void Bmm2ComputeCommon(const uint32_t bn2Idx, const
    // ↑ 换行续接 ↓
    uint32_t sInnerLoopIdx);
__aicore__ inline void SysPrefixBmm1Compute(const uint32_t bn2Idx, const
    // ↑ 换行续接 ↓
    uint32_t sInnerLoopIdx);
__aicore__ inline void SysPrefixBmm2Compute(const uint32_t bn2Idx, const
    // ↑ 换行续接 ↓
    uint32_t sInnerLoopIdx);

__aicore__ inline void DealBmm1ResBaseBlock(ExtraInfo& info, uint32_t startRow,
    // ↑ 换行续接 ↓
    uint32_t dealRowCount,
                                        uint32_t columnCount, uint32_t
                                           // ↑ 换行续接 ↓
                                        actualColumnCount);
__aicore__ inline void DealAntiqBmm1ResBaseBlock(ExtraInfo& info, uint32_t startRow,
                                           uint32_t dealRowCount,
                                              // ↑ 换行续接 ↓
                                              uint32_t columnCount,
                                           uint32_t actualColumnCount);
__aicore__ inline void DealAntiqBmm1ResBaseBlockPerToken(ExtraInfo&
    // ↑ 换行续接 ↓
    info, uint32_t startRow,
                                                 uint32_t dealRowCount,
                                                    // ↑ 换行续接 ↓
                                                    uint32_t columnCount,
                                                 uint32_t actualColumnCount);
__aicore__ inline void AntiquantMatmulResCombine(ExtraInfo& info, LocalTensor<T> bmmResUb,
    // ↑ 换行续接 ↓
    GlobalTensor<MM_OUT_T> srcGm,
                                           uint32_t startRow, uint32_t dealRowCount,
                                              // ↑ 换行续接 ↓
```

```
                                            uint32_t columnCount,
                                        uint32_t actualColumnCount);
    __aicore__ inline void AntiquantMatmulResCombineDD(ExtraInfo& info, LocalTensor<T> bmmResUb,
        // ↑ 换行续接 ↓
        GlobalTensor<MM_OUT_T> srcGm,
                                            uint32_t startRow, uint32_t dealRowCount,
                                                // ↑ 换行续接 ↓
                                                uint32_t columnCount,
                                            uint32_t actualColumnCount,
                                                // ↑ 换行续接 ↓
                                                float scaleC);
    __aicore__ inline void AntiquantMM2ResCombine(const ExtraInfo& info,
        // ↑ 换行续接 ↓
        LocalTensor<MM_OUT_T> bmmResUb, GlobalTensor<MM_OUT_T> srcGm,
                                            uint32_t startRow, uint32_t dealRowCount,
                                                // ↑ 换行续接 ↓
                                                uint32_t columnCount,
                                            uint32_t actualColumnCount);
    __aicore__ inline void ProcessVec1Inner(ExtraInfo& info);
    __aicore__ inline void PreProcessVec1(uint32_t sInnerLoopIdx);

    __aicore__ inline void DealAntiqBmm2ResBaseBlock(ExtraInfo& info, uint32_t startRow,
                                            uint32_t dealRowCount,
                                                // ↑ 换行续接 ↓
                                                uint32_t columnCount,
                                            uint32_t actualColumnCount);
    __aicore__ inline void DealAntiqBmm2ResBaseBlockPerToken(ExtraInfo& info,
        // ↑ 换行续接 ↓
        uint32_t startRow,
                                                    uint32_t dealRowCount,
                                                        // ↑ 换行续接 ↓
                                                        uint32_t columnCount,
                                                    uint32_t actualColumnCount);
    __aicore__ inline void ProcessVec2Inner(ExtraInfo& info);

    __aicore__ inline void SoftmaxFlashV2Compute(ExtraInfo& info, LocalTensor<T> &mmResUb,
        // ↑ 换行续接 ↓
        LocalTensor<uint8_t> &softmaxTmpUb,
                                        uint32_t startRow, uint32_t dealRowCount,
                                            // ↑ 换行续接 ↓
                                            uint32_t columnCount,
                                        uint32_t actualColumnCount);
    __aicore__ inline bool IsSkipAttenMask(const ExtraInfo &info, uint32_t startRow,
        // ↑ 换行续接 ↓
        uint32_t dealRowCount);
    __aicore__ inline void ElewiseCompute(ExtraInfo& info, LocalTensor<T> &mmResUb,
        // ↑ 换行续接 ↓
        TBuf<> &tmpBuf, uint32_t startRow,
                                    uint32_t dealRowCount, uint32_t columnCount,
                                        // ↑ 换行续接 ↓
                                    uint32_t actualColumnCount);

    __aicore__ inline void Bmm2DataCopyOut(uint64_t attenOutOffset, LocalTensor<OUT_T> &attenOutUb,
        // ↑ 换行续接 ↓
        uint32_t startRow, uint32_t dealRowCount,
                                     uint32_t columnCount, uint32_t
                                         // ↑ 换行续接 ↓
                                        actualColumnCount);
    __aicore__ inline void Bmm2DataCopyOutNcon(const ExtraInfo& info, LocalTensor<OUT_T> &attenOutUb,
        // ↑ 换行续接 ↓
        uint32_t startRow, uint32_t dealRowCount,
```

四、InitTilingData函数 (行585-638)

功能说明: 从TilingData读取所有参数

```
template <typename IFAT> __aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::InitTilingData()
{
    singleProcessSInnerSize = tilingData->increFlashAttentionSingleCoreParams.singleProcessSInnerSize;
    sInnerLoopTimes = tilingData->increFlashAttentionSingleCoreParams.sInnerLoopTimes;
    singleProcessSInnerSizeTail = tilingData->increFlashAttentionSingleCoreParams.singleProcessSInnerSizeTa
il;
    usedCoreNum = tilingData->increFlashAttentionSingleCoreParams.usedCoreNum;
    formerCoreNum = tilingData->increFlashAttentionSingleCoreParams.formerCoreNum;
```

```
    splitKVNum = tilingData->splitKVParams.s2;
    sInnerLoopSize = tilingData->splitKVParams.sInnerLoopSize;

    vec1ResUbSize = mmResUbSize * msdIterNum;
    mmResUbSize = tilingData->increFlashAttentionSingleCoreTensorSize.mmResUbSize;
    bmm2ResUbSize = tilingData->increFlashAttentionSingleCoreTensorSize.bmm2ResUbSize;

    batchSize = tilingData->baseParams.batchSize;
    kvHeadNum = tilingData->baseParams.kvHeadNum;
    qHeadNum = tilingData->baseParams.qHeadNum;
    gSize = tilingData->baseParams.nNumOfQInOneGroup;
    qSeqSize = tilingData->baseParams.qSeqSize;
    kvSeqSize = tilingData->baseParams.seqSize;
    headDim = tilingData->baseParams.headSize;
    batchContinuous = tilingData->baseParams.batchContinuousFlag;
    msdIterNum = tilingData->baseParams.msdIterNum;
    antiquantPerTensorFlag = tilingData->baseParams.antiquantPerTensorFlag;
    antiquantPerHeadFlag = (tilingData->baseParams.antiquantPerHeadFlag == 1);
    antiquantParamsInPagedAttentionFlag = (tilingData->baseParams.antiquantParamsInPagedAttentionFlag
        // ↑ 换行续接 ↓
        == 1);

    headDimAlign = Align(headDim, BYTE_BLOCK);

    gSizeSub = tilingData->increFlashAttentionSingleCoreParams.groupSplitSize; //切块大小Gi
    gOuter = (gSize + gSizeSub - 1) / gSizeSub;
    gSizeTail = gSize - (gOuter - 1) * gSizeSub;
    s1SizeSub = tilingData->increFlashAttentionSingleCoreParams.s1SplitSize; //切块大小Si
    s1Outer = (qSeqSize + s1SizeSub - 1) / s1SizeSub;
    s1SizeTail = qSeqSize - (s1Outer - 1) * s1SizeSub;

    attenMaskFlag = (tilingData->baseParams.attenMaskFlag != 0) ? true : false;
    attenMaskSize = tilingData->baseParams.attenMaskSize;
    selectWithByteMaskTmpMinSize = tilingData->baseParams.selectWithByteMaskTmpMinSize;

    antiqSeqSize = tilingData->baseParams.antiqSeqSize;

    kvPaddingFlag = tilingData->baseParams.kvPaddingFlag;

    // out quant
    isPerChnU8Out = tilingData->outputParams.isPerChnOut == 0 ? false : true;
    isOutQuantTypeBf16 = tilingData->outputParams.isOutQuantTypeBf16 == 0 ? false : true;

    // 是否输出lse
    softmaxLseFlag = tilingData->baseParams.softmaxLseFlag;

    maxBlockNumPerBatch = tilingData->baseParams.maxBlockNumPerBatch;
    kvCacheBlockSize = tilingData->baseParams.blockSize;
}
```

五、InitBuffers函数 (行640-692)

功能说明: 初始化所有UB/L1 Buffer

```
template <typename IFAT> __aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::InitBuffers()
{
    if ASCEND_IS_AIV {
        pipe->InitBuffer(inputBuf1, BUFFER_SIZE_BYTE_32K);
        pipe->InitBuffer(inputBuf2, BUFFER_SIZE_BYTE_16K);
        pipe->InitBuffer(outputBuf1, BUFFER_SIZE_BYTE_16K);
        pipe->InitBuffer(vec2ResBuff, BUFFER_SIZE_BYTE_16K);

        pipe->InitBuffer(outputBuf2, BUFFER_SIZE_BYTE_8K);

        // tmpBuff
        pipe->InitBuffer(tmpBuff1, BUFFER_SIZE_BYTE_32K);
        pipe->InitBuffer(tmpBuff2, BUFFER_SIZE_BYTE_32K);
        pipe->InitBuffer(tmpBuff3, BUFFER_SIZE_BYTE_2K);

        // 常驻buffer
        pipe->InitBuffer(antiqScaleBuff, BUFFER_SIZE_BYTE_4K);
        pipe->InitBuffer(antiqOffsetBuff, BUFFER_SIZE_BYTE_4K);
        // 预留空间2K = 64 * 32, 支持 gSize = 64
        // brcb 操作每次操作8*32字节输出, startRow接近64时,
```

```
        // 输出最多可能超出2k空间7*32字节， 这里预留256B防止越界
        pipe->InitBuffer(qAmaxBuff, Q_AMAX_BUF_BYTES * PRE_LOAD_NUM_DD);
        pipe->InitBuffer(softmaxResAmaxBuff, P_AMAX_BUF_BYTES * PRE_LOAD_NUM_DD)
            // ↑ 换行续接 ↓
            ; // perToken使用,perChannel不需要
        pipe->InitBuffer(softmaxMaxBuff, SOFTMAX_MAX_BUF_BYTES * PRE_LOAD_NUM_DD);
        pipe->InitBuffer(softmaxExpBuff, SOFTMAX_EXP_BUF_BYTES * PRE_LOAD_NUM_DD);
        pipe->InitBuffer(softmaxSumBuff, SOFTMAX_SUM_BUF_BYTES * PRE_LOAD_NUM_DD);
#ifdef IFA_SOFTMAX_WITHOUT_BRC_PRELOAD
        pipe->InitBuffer(softmaxMaxDefaultBuff, SOFTMAX_MAX_BUF_BYTES);
        pipe->InitBuffer(softmaxSumDefaultBuff, SOFTMAX_SUM_BUF_BYTES);
#else
        pipe->InitBuffer(softmaxMaxDefaultBuff, BUFFER_SIZE_BYTE_512B);
        pipe->InitBuffer(softmaxSumDefaultBuff, BUFFER_SIZE_BYTE_512B);
#endif
    } else {

        // L1
        pipe->InitBuffer(tmpBufL1Q, L1Q_BLOCK_SIZE);
        qL1Buffers = tmpBufL1Q.Get<KV_T>();
        pipe->InitBuffer(tmpBufL1KP, 3 * L1KP_BLOCK_SIZE);
        kpL1Buffers = tmpBufL1KP.Get<KV_T>();
        pipe->InitBuffer(tmpBufL1V, 4 * L1V_BLOCK_SIZE);
        vL1Buffers = tmpBufL1V.Get<KV_T>();

        pipe->InitBuffer(tmpBufL0A, 2 * L0A_PP_SIZE);
        aL0TensorPingPong = tmpBufL0A.Get<KV_T>();
        // L0B
        pipe->InitBuffer(tmpBufL0B, 2 * L0B_PP_SIZE);
        bL0TensorPingPong = tmpBufL0B.Get<KV_T>();
        // L0C
        pipe->InitBuffer(tmpBufL0C, 2 * L0C_PP_SIZE);
        cL0TensorPingPong = tmpBufL0C.Get<L0C_T>();
    }
}
```

六、UpdateInnerLoopCond函数 (行815-843)

功能说明: 动态计算S方向切分参数

```
template <typename IFAT> __aicore__ inline void
IncreFlashAttentionAttenPreloadDD<IFAT>::UpdateInnerLoopCond()
{
    if (curActualSeqLen == 0) {
        curActSeqLenIsZero = true;
        return;
    }
    curActSeqLenIsZero = false;

    int32_t remainSinnerSize = (int32_t)curActualSeqLen;
    int32_t computeSinnerSize = (int32_t)curActualSeqLen;
    if constexpr (FLASH_DECODE) {
        remainSinnerSize = (int32_t)curActualSeqLen - sInnerLoopSize * s2IdxFD;
        computeSinnerSize = remainSinnerSize >= sInnerLoopSize ? sInnerLoopSize
            // ↑ 换行续接 ↓
            : remainSinnerSize;
        if (aiCoreIdx >= batchSize * kvHeadNum * splitKVNum) {
            remainSinnerSize = 0;
        }
    }
    if (remainSinnerSize > 0) {
        if (computeSinnerSize <= singleProcessSInnerSize) {
            singleProcessSInnerSizeTail = computeSinnerSize;
            sInnerLoopTimes = 1;
        } else {
            sInnerLoopTimes = (computeSinnerSize + singleProcessSInnerSize - 1)
                // ↑ 换行续接 ↓
                / singleProcessSInnerSize;
            singleProcessSInnerSizeTail = computeSinnerSize - (sInnerLoopTimes - 1)
                // ↑ 换行续接 ↓
                * singleProcessSInnerSize;
        }
    } else {
        sInnerLoopTimes = 0;
```

```
    }
}
```

七、Init主初始化函数 (行882-1004)

功能说明: Kernel主初始化流程（Workspace分配）

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::Init(
    __gm__ uint8_t *query, __gm__ uint8_t *key, __gm__ uint8_t *value, __gm__
        // ↑ 换行续接 ↓
        uint8_t *pseShift,
    __gm__ uint8_t *attenMask, __gm__ uint8_t *actualSeqLengthsQ, __gm__
        // ↑ 换行续接 ↓
        uint8_t *actualSeqLengths,
    __gm__ uint8_t *blockTable, __gm__ uint8_t *kvPaddingSize, __gm__ uint8_t
        // ↑ 换行续接 ↓
        *attentionOut,
    __gm__ uint8_t *softmaxLse, __gm__ uint8_t *workspace, const IncreFlashAttentionTilingData
        // ↑ 换行续接 ↓
        *__restrict tiling,
    __gm__ uint8_t *gmTiling, TPipe *tPipe, bool isPrefix)
{
    if ASCEND_IS_AIV {
        tmpBlockIdx = GetBlockIdx(); // vec:0-47
        aiCoreIdx = tmpBlockIdx / 2;
    } else {
        tmpBlockIdx = GetBlockIdx(); // cube:0-23
        aiCoreIdx = tmpBlockIdx;
    }
    // init tiling data
    tilingData = tiling;

    InitTilingData();
    // 初始化计算参数
    if constexpr (FLASH_DECODE) {
        InitCalcParams();
    } else {
        InitCalcParamsEach();
    }

    pipe = tPipe;
    keyPtr = key;
    valuePtr = value;
    blocktablePtr = blockTable;

    // init global buffer
    queryGm.SetGlobalBuffer((__gm__ Q_T *)query);
    attentionOutGm.SetGlobalBuffer((__gm__ OUT_T *)attentionOut);
    // batch连续时,只需要初始化一次;不连续时,需要在使用时根据batchIdx初始化
    if (batchContinuous) {
        InitKeyGm(0);
        InitValueGm(0);
    }

    if (pipe != nullptr) {
        InitBuffers();
    }

    if (attenMaskFlag) {
        attenMaskBoolGm.SetGlobalBuffer((__gm__ bool *)attenMask);
    }

    InitActualSeqLen(actualSeqLengthsQ, actualSeqLengths);

    if (kvPaddingFlag == 1) {
        kvPaddingSizeGm.SetGlobalBuffer((__gm__ int64_t *)kvPaddingSize);
    }

    if constexpr (PAGE_ATTENTION) {
        blockTableGm.SetGlobalBuffer((__gm__ int32_t *)blockTable);
    }

    if ASCEND_IS_AIV {
        softmaxMaxUb = softmaxMaxBuff.Get<T>();
```

```
    softmaxSumUb = softmaxSumBuff.Get<T>();
    softmaxExpUb = softmaxExpBuff.Get<T>();
    softmaxMaxDefaultUb = softmaxMaxDefaultBuff.Get<T>();
    softmaxSumDefaultUb = softmaxSumDefaultBuff.Get<T>();
}

uint64_t offset = 0;
if constexpr (ANTIQUANT) {
    if constexpr (KVINT4) {
        queryPreProcessResGm.SetGlobalBuffer(
            (__gm__ KV_T *)(workspace + offset + (aiCoreIdx * dbWorkspaceRatio
                // ↑ 换行续接 ↓
                * bmm2ResUbSize * sizeof(KV_T) >> 1)));
        offset += (GetBlockNum() * dbWorkspaceRatio * bmm2ResUbSize
            // ↑ 换行续接 ↓
            * sizeof(KV_T) >> 1);
    } else {
        queryPreProcessResGm.SetGlobalBuffer(
            (__gm__ KV_T *)(workspace + offset + aiCoreIdx * dbWorkspaceRatio
                // ↑ 换行续接 ↓
                * bmm2ResUbSize * sizeof(KV_T)));
        offset += GetBlockNum() * dbWorkspaceRatio * bmm2ResUbSize * sizeof(KV_T);
    }
}

mm1ResGm.SetGlobalBuffer(
       (__gm__ MM_OUT_T *)(workspace + offset + aiCoreIdx * dbWorkspaceRatio
            // ↑ 换行续接 ↓
            * mmResUbSize * sizeof(MM_OUT_T)));
offset += GetBlockNum() * dbWorkspaceRatio * mmResUbSize * sizeof(MM_OUT_T);

if constexpr (KVINT4) {
    vec1ResGm.SetGlobalBuffer(
        (__gm__ KV_T *)(workspace + offset + (aiCoreIdx * dbWorkspaceRatio
            // ↑ 换行续接 ↓
            * mmResUbSize * sizeof(KV_T) >> 1)));
    offset += (GetBlockNum() * dbWorkspaceRatio * mmResUbSize * sizeof(KV_T) >> 1);
} else {
    vec1ResGm.SetGlobalBuffer(
        (__gm__ KV_T *)(workspace + offset + aiCoreIdx * dbWorkspaceRatio
            // ↑ 换行续接 ↓
            * mmResUbSize * sizeof(KV_T)));
    offset += GetBlockNum() * dbWorkspaceRatio * mmResUbSize * sizeof(KV_T);
}


mm2ResGm.SetGlobalBuffer(
       (__gm__ MM_OUT_T *)(workspace + offset + aiCoreIdx * dbWorkspaceRatio
            // ↑ 换行续接 ↓
            * bmm2ResUbSize * sizeof(MM_OUT_T)));
offset += GetBlockNum() * dbWorkspaceRatio * bmm2ResUbSize * sizeof(MM_OUT_T);

if constexpr (ANTIQUANT) {
    vec2ResGm.SetGlobalBuffer(
           (__gm__ MM_OUT_T *)(workspace + offset + aiCoreIdx * dbWorkspaceRatio*
               // ↑ 换行续接 ↓
               bmm2ResUbSize * sizeof(MM_OUT_T)));
    offset += GetBlockNum() * dbWorkspaceRatio * bmm2ResUbSize * sizeof(MM_OUT_T);
} else {
    vec2ResGm.SetGlobalBuffer(
           (__gm__ MM_OUT_T *)(workspace + offset + aiCoreIdx * dbWorkspaceRatio*
               // ↑ 换行续接 ↓
               bmm2ResUbSize * sizeof(T)));
    offset += GetBlockNum() * dbWorkspaceRatio * bmm2ResUbSize * sizeof(MM_OUT_T);
}
offset += GetBlockNum() * dbWorkspaceRatio * bmm2ResUbSize * sizeof(T);

// GM for pse


if constexpr (FLASH_DECODE) {
    accumOutGm.SetGlobalBuffer((__gm__ float *)(workspace + offset));
    offset = offset + tilingData->splitKVParams.accumOutSize * sizeof(float);
    lseSumFdGm.SetGlobalBuffer((__gm__ float *)(workspace + offset));
    lseMaxFdGm.SetGlobalBuffer((__gm__ float *)(workspace + offset)
        // ↑ 换行续接 ↓
        + tilingData->splitKVParams.logSumExpSize / 2);
    offset = offset + tilingData->splitKVParams.logSumExpSize * sizeof(float);
}
```

```
    if (softmaxLseFlag) {
        softmaxLseGm.SetGlobalBuffer((__gm__ float *)softmaxLse);
    }
}
```

八、AttenMaskCopyIn函数 (行1168-1227)

功能说明: Attention Mask加载和应用

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::AttenMaskCopyIn(uint64_t
    // ↑ 换行续接 ↓
    offset,

                                                                                // ↑ 换行续接 ↓
                                                                                uint32_t dealRowCount,

                                                                                // ↑ 换行续接 ↓
                                                                                uint32_t actualColumnCount)
{
    LocalTensor<bool> maskUb = inputBuf1.Get<bool>();
    attenMaskSizeAlign = Align(actualColumnCount, 32U);
    maskUb.SetSize(dealRowCount * attenMaskSizeAlign);
    WaitFlag<AscendC::HardEvent::V_MTE2>(SYNC_INPUT_BUF1_FLAG);
#if (__CCE_AICORE__ > 200)
    if (actualColumnCount % 32 == 0) {
        DataCopy(maskUb, attenMaskBoolGm[offset], attenMaskSizeAlign);
    } else {
        uint32_t typeElementSize = BYTE_BLOCK / sizeof(bool);
        DataCopyExtParams intriParams;
        intriParams.blockLen = actualColumnCount * sizeof(bool);
        intriParams.blockCount = 1;
        intriParams.dstStride = (attenMaskSizeAlign - actualColumnCount)
            // ↑ 换行续接 ↓
            / typeElementSize;
        intriParams.srcStride = 0;
        DataCopyPadExtParams<bool> padParams;
        padParams.isPad = true;
        padParams.leftPadding = 0;
        padParams.rightPadding = (attenMaskSizeAlign - actualColumnCount)
            // ↑ 换行续接 ↓
            % typeElementSize;
        padParams.paddingValue = 0;
        DataCopyPad(maskUb, attenMaskBoolGm[offset], intriParams, padParams);
    }
#else
    DataCopy(maskUb, attenMaskBoolGm[offset], attenMaskSizeAlign);
#endif
    SetFlag<AscendC::HardEvent::MTE2_V>(SYNC_INPUT_BUF1_FLAG);
}

template <typename IFAT>
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::AttenMaskCopyIn(
    // ↑ 换行续接 ↓
    const ExtraInfo& info)
{
    uint32_t offset;
    {
        int32_t delta =
            info.s1Idx * s1SizeSub - info.s2Idx * singleProcessSInnerSize
                // ↑ 换行续接 ↓
                + info.s2Size - info.actSeqLenQ; // s1idx = 0
        if (delta < 0) {
            offset = (-delta) < (int32_t)info.s1Size ? (-delta) : info.s1Size;
                // ↑ 换行续接 ↓
                //min (-delta, s1Size)
        } else {
            offset = (delta < (int32_t)singleProcessSInnerSize ? delta :
                // ↑ 换行续接 ↓
                singleProcessSInnerSize) *
                    ATTENMASK_STRIDE; // min(delta, s2inner)
        }
    }

```

```
    attenMaskSizeAlign = Align(info.actualSingleProcessSInnerSize, 32U);

    DataCopyParams dataCopyParams;
    dataCopyParams.blockCount = info.s1Size;
    dataCopyParams.blockLen = attenMaskSizeAlign * sizeof(bool) / 32;
    dataCopyParams.srcStride = (ATTENMASK_STRIDE - attenMaskSizeAlign) *
        // ↑ 换行续接 ↓
        sizeof(bool) / 32;
    dataCopyParams.dstStride = 0;

    LocalTensor<bool> maskUb = inputBuf1.Get<bool>();
    WaitFlag<AscendC::HardEvent::V_MTE2>(SYNC_INPUT_BUF1_FLAG);
    DataCopy(maskUb, attenMaskBoolGm[offset], dataCopyParams);

    SetFlag<AscendC::HardEvent::MTE2_V>(SYNC_INPUT_BUF1_FLAG);
}
```

九、ComputeMm1函数 (行3065-3299)

功能说明: Q×K^T矩阵乘法完整流程

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::ComputeMm1(ExtraInfo&
    // ↑ 换行续接 ↓
    info) {
    LocalTensor<KV_T> aL1Tensor = qL1Buffers[qL1BufIter * L1Q_BLOCK_SIZE / sizeof(KV_T)];
    if (info.s2Idx == 0) {
        WaitFlag<HardEvent::MTE1_MTE2>(L1Q_EVENT0 + qL1BufIter);
        CopyInMm1AToL1(aL1Tensor, info);
        SetFlag<HardEvent::MTE2_MTE1>(L1Q_EVENT0 + qL1BufIter);
        WaitFlag<HardEvent::MTE2_MTE1>(L1Q_EVENT0 + qL1BufIter);
    }
    constexpr uint32_t nCopyRowCount = KV_LOAD_TO_L1_ROW_NUM;
    uint32_t nCopyTimes = (info.actualSingleProcessSInnerSize + nCopyRowCount
        // ↑ 换行续接 ↓
        - 1) / nCopyRowCount;
    uint32_t nTailCopyRowCount = info.actualSingleProcessSInnerSize - (nCopyTimes
        // ↑ 换行续接 ↓
        - 1) * nCopyRowCount;
    uint32_t nTailCopyRowCountAlign = info.actualSingleProcessSInnerSizeAlign - (
        // ↑ 换行续接 ↓
        nCopyTimes - 1) * nCopyRowCount;
    for (uint32_t nCopyIdx = 0, nActCopyRowCount = nCopyRowCount, nActCopyRowCountAlign = nCopyRowCount;
        // ↑ 换行续接 ↓
        nCopyIdx < nCopyTimes; nCopyIdx++) {
        if (nCopyIdx + 1 == nCopyTimes) {
            nActCopyRowCount = nTailCopyRowCount;
            nActCopyRowCountAlign = nTailCopyRowCountAlign;
        }
        // A:256*128 B:1024*128 A*B^T INT8  M=256 N=1024 K=128
        // A全载到L1，B的N方向按照512切分，计算2次
        kpL1BufIter++;
        LocalTensor<KV_T> bL1Tensor = kpL1Buffers[(kpL1BufIter % 3) * L1KP_BLOCK_SIZE
            // ↑ 换行续接 ↓
            / sizeof(KV_T)];
        WaitFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT0 + (kpL1BufIter % 3));

        if constexpr (PAGE_ATTENTION) {
            uint64_t blockTableBaseOffset = info.bIdx * maxBlockNumPerBatch;
            uint32_t curSeqIdx = info.s2BatchOffset + nCopyIdx * nCopyRowCount;
            uint32_t copyFinishRowCnt = 0;
            while (copyFinishRowCnt < nActCopyRowCount) {
                uint64_t blockIdOffset = curSeqIdx / kvCacheBlockSize; //
                    // ↑ 换行续接 ↓
                    获取blcok table上的索引
                uint64_t reaminRowCnt = curSeqIdx % kvCacheBlockSize;  // 获取在单个块上超出的行数
                uint64_t idInBlockTable =
                    blockTableGm.GetValue(blockTableBaseOffset + blockIdOffset)
                        // ↑ 换行续接 ↓
                        ; // 从block table上的获取编号
                // 计算可以拷贝行数
                uint32_t copyRowCnt = kvCacheBlockSize - reaminRowCnt;
                if (copyFinishRowCnt + copyRowCnt > nActCopyRowCount) {
                    copyRowCnt = nActCopyRowCount - copyFinishRowCnt;
                }
```

```
            uint64_t keyOffset = idInBlockTable * kvCacheBlockSize *
                // ↑ 换行续接 ↓
                headDim * kvHeadNum;
            if constexpr (KV_LAYOUT_T == LAYOUT::NZ) {
                uint32_t blockElementCnt = 32 / sizeof(KV_T);
                if constexpr (KVINT4) {
                    blockElementCnt = 64;
                }
                keyOffset += (uint64_t)(info.n2Idx * headDim * kvCacheBlockSize)
                    // ↑ 换行续接 ↓
                    + reaminRowCnt * blockElementCnt;
            } else {
                if constexpr (KV_LAYOUT_T == LAYOUT::BSH || KV_LAYOUT_T
                    // ↑ 换行续接 ↓
                    == LAYOUT::BSND) {
                    keyOffset += (uint64_t)(info.n2Idx * headDim) +
                        // ↑ 换行续接 ↓
                        reaminRowCnt * headDim * kvHeadNum;
                } else {
                    keyOffset += (uint64_t)(info.n2Idx * headDim * kvCacheBlockSize)
                        // ↑ 换行续接 ↓
                        + reaminRowCnt * headDim;
                }
            }
            CopyInMm1BToL1ForPA(bL1Tensor, keyOffset, nActCopyRowCountAlign,
                // ↑ 换行续接 ↓
                copyFinishRowCnt, copyRowCnt);

            // 更新循环变量
            copyFinishRowCnt += copyRowCnt;
            curSeqIdx += copyRowCnt;
        }
    } else {
        CopyInMm1BToL1(bL1Tensor, info, nCopyIdx, nCopyRowCount, nActCopyRowCount,
            // ↑ 换行续接 ↓
            nActCopyRowCountAlign);
    }

    SetFlag<HardEvent::MTE2_MTE1>(L1KP_EVENT0 + (kpL1BufIter % 3));
    WaitFlag<HardEvent::MTE2_MTE1>(L1KP_EVENT0 + (kpL1BufIter % 3));

    // A:256*128，M方向按照64循环
    constexpr uint32_t baseM = 64 / sizeof(KV_T);
    uint32_t mActCopyRowCount = msdIterNum * info.mSize;
    uint32_t mLoopTimes = (mActCopyRowCount + baseM - 1) / baseM;
    uint32_t mTail = mActCopyRowCount - (mLoopTimes - 1) * baseM;
    bool isHead = false;
    bool isMid = false;
    bool isTail = false;
    bool isOdd = (mLoopTimes % 2) != 0 ? true : false;
    bool hasTail = (mTail == baseM) ? false : true;
    for (uint32_t i = 0, actualBaseM = baseM; i < mLoopTimes; i++) {
        if (i + 1 == mLoopTimes) {
            actualBaseM = mTail;
        }
        isHead = (!isOdd && ((!hasTail && (i < (mLoopTimes / 2))) || (
            // ↑ 换行续接 ↓
            i < (mLoopTimes / 2 - 1)))) || (isOdd && (i < (mLoopTimes / 2)));
        isMid = (!isOdd && hasTail && (i == (mLoopTimes / 2 - 1))) || (
            // ↑ 换行续接 ↓
            isOdd && (i == (mLoopTimes / 2)));
        isTail = (!isOdd && (!hasTail || (i > (mLoopTimes / 2 - 1)))
            // ↑ 换行续接 ↓
            ) || (isOdd && (i > (mLoopTimes / 2)));
        LocalTensor<KV_T> aL0Tensor = aL0TensorPingPong[(aL0BufIter % 2)
            // ↑ 换行续接 ↓
            * L0A_PP_SIZE / sizeof(KV_T)];
        WaitFlag<HardEvent::M_MTE1>(L0A_EVENT0 + (aL0BufIter % 2));
        LoadData2DParams loadData2DParamsForA;
        loadData2DParamsForA.startIndex = 0;
        if constexpr (KVINT4) {
            loadData2DParamsForA.repeatTimes = (actualBaseM + 15) /
                // ↑ 换行续接 ↓
                16 * headDimAlign / 64;
        } else {
            loadData2DParamsForA.repeatTimes = (actualBaseM + 15) /
                // ↑ 换行续接 ↓
                16 * headDimAlign / (32 / sizeof(KV_T));
        }
```

```
            loadData2DParamsForA.srcStride = 1;
            loadData2DParamsForA.dstGap = 0;
            loadData2DParamsForA.ifTranspose = false;
            LoadData(aL0Tensor, aL1Tensor[i * baseM * headDimAlign], loadData2DParamsForA);
            SetFlag<HardEvent::MTE1_M>(L0A_EVENT0 + (aL0BufIter % 2));
            WaitFlag<HardEvent::MTE1_M>(L0A_EVENT0 + (aL0BufIter % 2));

            constexpr uint32_t baseN = 256 / sizeof(KV_T);
            uint32_t nLoopTimes = (nActCopyRowCountAlign + baseN - 1) / baseN;
            uint32_t nTail = nActCopyRowCountAlign - (nLoopTimes - 1) * baseN;
            for (uint32_t j = 0, actualBaseN = baseN; j < nLoopTimes; j++) {
                if (j + 1 == nLoopTimes) {
                    actualBaseN = nTail;
                }
                LocalTensor<KV_T> bL0Tensor = bL0TensorPingPong[(bL0BufIter % 2)
                    // ↑ 换行续接 ↓
                    * L0B_PP_SIZE / sizeof(KV_T)];
                WaitFlag<HardEvent::M_MTE1>(L0B_EVENT0 + (bL0BufIter % 2));
                uint32_t blockElementCnt = 32 / sizeof(KV_T);
                if constexpr (KVINT4) {
                    blockElementCnt = 64;
                }
                LoadData2DParams loadData2DParamsForB;
                loadData2DParamsForB.startIndex = 0;
                loadData2DParamsForB.srcStride = 1;
                loadData2DParamsForB.dstGap = 0;
                loadData2DParamsForB.ifTranspose = false;
#ifdef L1_LAYOUT_zN
                loadData2DParamsForB.repeatTimes = actualBaseN / 16;
                uint32_t loadLoopTimes = headDimAlign / blockElementCnt;
                for(uint32_t k = 0; k < loadLoopTimes; k++){
                    LoadData(bL0Tensor[k * actualBaseN * blockElementCnt],
                        bL1Tensor[(j * baseN + k * nActCopyRowCountAlign)
                            // ↑ 换行续接 ↓
                            * blockElementCnt], loadData2DParamsForB);
                }
#else
                loadData2DParamsForB.repeatTimes = (actualBaseN / 16) * (
                    // ↑ 换行续接 ↓
                    headDimAlign / blockElementCnt);
                LoadData(bL0Tensor, bL1Tensor[baseN * headDimAlign * j],
                    // ↑ 换行续接 ↓
                    loadData2DParamsForB);
#endif
                SetFlag<HardEvent::MTE1_M>(L0B_EVENT0 + (bL0BufIter % 2));
                WaitFlag<HardEvent::MTE1_M>(L0B_EVENT0 + (bL0BufIter % 2));

                MmadParams mmadParams;
                mmadParams.m = actualBaseM;
                if (mmadParams.m == 1) {  // m等于1会默认开GEMV模式，且不可关闭GEMV，所以规避当作矩阵计算
                    mmadParams.m = 16;
                }
                mmadParams.n = actualBaseN; // 无效数据不参与计算
                mmadParams.k = 128;
                mmadParams.cmatrixInitVal = true;
                mmadParams.cmatrixSource = false;

                LocalTensor<L0C_T> cL0Tensor = cL0TensorPingPong[(cL0BufIter % 2)
                    // ↑ 换行续接 ↓
                    * L0C_PP_SIZE / sizeof(L0C_T)];
                WaitFlag<HardEvent::FIX_M>(L0C_EVENT0 + (cL0BufIter % 2));

                Mmad(cL0Tensor, aL0Tensor, bL0Tensor, mmadParams);
                PipeBarrier<PIPE_M>();
                SetFlag<HardEvent::M_FIX>(L0C_EVENT0 + (cL0BufIter % 2));
                WaitFlag<HardEvent::M_FIX>(L0C_EVENT0 + (cL0BufIter % 2));

                if (mLoopTimes == 1) {
                    for (uint32_t mIter = 0; mIter < msdIterNum; mIter++) {
                        float tmp = quantScaleC1S1;
                        if (mIter == 1) {
                            tmp = quantScaleC1S2;
                        }
                        FixpipeParamsV220 fixParams;
                        fixParams.nSize = actualBaseN;
                        fixParams.mSize = actualBaseM / msdIterNum; // 有效数据不足16行，只需要输出部分行即可
                        fixParams.srcStride = ((actualBaseM + 15) / 16) * 16;
                        fixParams.dstStride = info.actualSingleProcessSInnerSizeAlign;

```

```
                        // ↑ 换行续接 ↓
                        // mm1ResGm两行之间的间隔
                    fixParams.ndNum = 1;
                    fixParams.quantPre = QuantMode_t::DEQF16;
                    fixParams.deqScalar = static_cast<uint64_t>(*reinterpret_cast<int32_t
                        // ↑ 换行续接 ↓
                        *>(&tmp));
                    if (mIter == 1) {
                        SetAtomicAdd<half>();
                    }
                    Fixpipe(mm1ResGm[(info.loop % (PRE_LOAD_NUM_DD))
                        // ↑ 换行续接 ↓
                        * mmResUbSize + i * baseM * info.actualSingleProcessSInnerSizeAlign + nCopyIdx * nCop
yRowCount + j * baseN],
                            cL0Tensor[info.mSize * mIter * 16], fixParams);
                    if (mIter == 1) {
                        SetAtomicNone();
                    }
                    PipeBarrier<PIPE_FIX>();
                }
            } else if (isHead) {
                float tmp = quantScaleC1S1;
                FixpipeParamsV220 fixParams;
                fixParams.nSize = actualBaseN;
                fixParams.mSize = actualBaseM; // 有效数据不足16行，只需要输出部分行即可
                fixParams.srcStride = ((actualBaseM + 15) / 16) * 16;
                fixParams.dstStride = info.actualSingleProcessSInnerSizeAlign;
                    // ↑ 换行续接 ↓
                    // mm1ResGm两行之间的间隔
                fixParams.ndNum = 1;
                fixParams.quantPre = QuantMode_t::DEQF16;
                fixParams.deqScalar = static_cast<uint64_t>(*reinterpret_cast<int32_t
                    // ↑ 换行续接 ↓
                    *>(&tmp));
                Fixpipe(mm1ResGm[(info.loop % (PRE_LOAD_NUM_DD)) * mmResUbSize
                    // ↑ 换行续接 ↓
                    + i * baseM * info.actualSingleProcessSInnerSizeAlign + nCopyIdx * nCopyRowCount + j * b
aseN],
                        cL0Tensor, fixParams);
                PipeBarrier<PIPE_FIX>();
            } else if (isTail) {
                float tmp = quantScaleC1S2;
                FixpipeParamsV220 fixParams;
                fixParams.nSize = actualBaseN;
                fixParams.mSize = actualBaseM; // 有效数据不足16行，只需要输出部分行即可
                fixParams.srcStride = ((actualBaseM + 15) / 16) * 16;
                fixParams.dstStride = info.actualSingleProcessSInnerSizeAlign;
                    // ↑ 换行续接 ↓
                    // mm1ResGm两行之间的间隔
                fixParams.ndNum = 1;
                fixParams.quantPre = QuantMode_t::DEQF16;
                fixParams.deqScalar = static_cast<uint64_t>(*reinterpret_cast<int32_t
                    // ↑ 换行续接 ↓
                    *>(&tmp));
                SetAtomicAdd<half>();
                Fixpipe(mm1ResGm[(info.loop % (PRE_LOAD_NUM_DD)) * mmResUbSize + (
                    // ↑ 换行续接 ↓
                    i * baseM - info.mSize) * info.actualSingleProcessSInnerSizeAlign + nCopyIdx * nCopyRowC
ount + j * baseN],
                        cL0Tensor, fixParams);
                SetAtomicNone();
                PipeBarrier<PIPE_FIX>();
            } else {
                float tmp = quantScaleC1S1;
                FixpipeParamsV220 fixParams;
                fixParams.nSize = actualBaseN;
                fixParams.mSize = info.mSize - i * baseM; // 有效数据不足16行，只需要输出部分行即可
                fixParams.srcStride = ((actualBaseM + 15) / 16) * 16;
                fixParams.dstStride = info.actualSingleProcessSInnerSizeAlign;
                    // ↑ 换行续接 ↓
                    // mm1ResGm两行之间的间隔
                fixParams.ndNum = 1;
                fixParams.quantPre = QuantMode_t::DEQF16;
                fixParams.deqScalar = static_cast<uint64_t>(*reinterpret_cast<int32_t
                    // ↑ 换行续接 ↓
                    *>(&tmp));
                Fixpipe(mm1ResGm[(info.loop % (PRE_LOAD_NUM_DD)) * mmResUbSize
                    // ↑ 换行续接 ↓
                    + i * baseM * info.actualSingleProcessSInnerSizeAlign + nCopyIdx * nCopyRowCount + j * b
```

```
aseN],
                            cL0Tensor, fixParams);
                    PipeBarrier<PIPE_FIX>();
                    tmp = quantScaleC1S2;
                    fixParams.mSize = (i + 1) * baseM - info.mSize;
                    fixParams.deqScalar = static_cast<uint64_t>(*reinterpret_cast<int32_t
                        // ↑ 换行续接 ↓
                        *>(&tmp));
                    SetAtomicAdd<half>();
                    Fixpipe(mm1ResGm[(info.loop % (PRE_LOAD_NUM_DD)) * mmResUbSize + nCopyIdx * nCopyRowCount +
j * baseN],
                        // ↑ 换行续接 ↓
                        cL0Tensor[(info.mSize - i * baseM) * 16], fixParams);
                    SetAtomicNone();
                    PipeBarrier<PIPE_FIX>();
                }
                SetFlag<HardEvent::FIX_M>(L0C_EVENT0 + (cL0BufIter % 2));
                cL0BufIter++;
                SetFlag<HardEvent::M_MTE1>(L0B_EVENT0 + (bL0BufIter % 2));
                bL0BufIter++;
            }
            SetFlag<HardEvent::M_MTE1>(L0A_EVENT0 + (aL0BufIter % 2));
            aL0BufIter++;
        }
        SetFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT0 + (kpL1BufIter % 3));
    }
    // will change batch
    if ((info.s2Idx + 1) == info.curSInnerLoopTimes) {
        SetFlag<HardEvent::MTE1_MTE2>(L1Q_EVENT0 + qL1BufIter);
    }
}
```

十、ProcessVec1Inner函数 (行2354-2395)

功能说明: MM1结果的Softmax归一化

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::ProcessVec1Inner(ExtraInfo&
    // ↑ 换行续接 ↓
    info)
{
    uint32_t mSplitSize = BASE_BLOCK_MAX_ELEMENT_NUM / info.actualSingleProcessSInnerSizeAlign;
#ifdef IFA_SOFTMAX_WITHOUT_BRC_PRELOAD
    // 1. 向下8对齐是因为UB操作至少32B
    // 2. info.actualSingleProcessSInnerSizeAlign最大512，mSplitSize可以确保最小为16
    mSplitSize = mSplitSize / 8 * 8;
#endif
    if (mSplitSize > mSizeVector) {
        mSplitSize = mSizeVector;
    }
    uint32_t loopCount = (mSizeVector + mSplitSize - 1) / mSplitSize;
    uint32_t tailSplitSize = mSizeVector - (loopCount - 1) * mSplitSize;

    for (uint32_t i = 0, dealSize = mSplitSize; i < loopCount; i++) {
        if (i == (loopCount - 1)) {
            dealSize = tailSplitSize;
        }
        if constexpr (ANTIQUANT) {
            if constexpr (ANTIQUANT_PER_CHANNEL) {
                DealAntiqBmm1ResBaseBlock(info, i * mSplitSize, dealSize,
                    // ↑ 换行续接 ↓
                    info.actualSingleProcessSInnerSizeAlign,
                                    info.actualSingleProcessSInnerSize);
            } else if constexpr (ANTIQUANT_PER_TOKEN) {
                DealAntiqBmm1ResBaseBlockPerToken(info, i * mSplitSize, dealSize,
                                                  info.actualSingleProcessSInnerSizeAlign,
                                                      // ↑ 换行续接 ↓
                                                      info.actualSingleProcessSInnerSize);
            }
        } else {
            DealBmm1ResBaseBlock(info, i * mSplitSize, dealSize, info.actualSingleProcessSInnerSizeAlign,
                                 info.actualSingleProcessSInnerSize);
        }
    }

```

```
    if (info.s2Idx == sInnerLoopTimes - 1) {
        if constexpr (FLASH_DECODE) {
            uint32_t outIdx = info.loop % (PRE_LOAD_NUM_DD);
            auto sumTensor = softmaxSumUb[outIdx * SOFTMAX_SUM_BUF_SIZE];
            auto maxTensor = softmaxMaxUb[outIdx * SOFTMAX_MAX_BUF_SIZE];
            ComputeLogSumExpAndCopyToGm(sumTensor, maxTensor);
            return;
        }
    }
}
```

十一、SoftmaxFlashV2Compute函数 (行2009-2045)

功能说明: Flash Attention V2 Softmax核心

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::SoftmaxFlashV2Compute(ExtraInfo&
    // ↑ 换行续接 ↓
    info,
    LocalTensor<T> &mmResUb, LocalTensor<uint8_t> &softmaxTmpUb, uint32_t startRow,
        // ↑ 换行续接 ↓
        uint32_t dealRowCount,
    uint32_t columnCount, uint32_t actualColumnCount)
{
#ifdef IFA_SOFTMAX_WITHOUT_BRC_PRELOAD
    uint32_t baseOffset = startRow;
#else
    uint32_t baseOffset = startRow * BLOCK_ELEMENT_NUM;
#endif
    SoftMaxShapeInfo srcShape{dealRowCount, columnCount, dealRowCount, actualColumnCount};
    SoftMaxTiling newTiling =
        SoftMaxFlashV2TilingFunc(srcShape, sizeof(T), sizeof(T), softmaxTmpUb.GetSize(),
            // ↑ 换行续接 ↓
            true, false);

    LocalTensor<T> inSumTensor;
    LocalTensor<T> inMaxTensor;
    uint32_t outIdx = info.loop % (PRE_LOAD_NUM_DD);
    if (info.isFirstSInnerLoop) {
        inMaxTensor = softmaxMaxDefaultUb;
        inSumTensor = softmaxSumDefaultUb;
    } else {
        uint32_t inIdx = (info.loop - 1) % (PRE_LOAD_NUM_DD);
        inMaxTensor = softmaxMaxUb[inIdx * SOFTMAX_MAX_BUF_SIZE + baseOffset];
        inSumTensor = softmaxSumUb[inIdx * SOFTMAX_SUM_BUF_SIZE + baseOffset];
    }

#ifdef IFA_SOFTMAX_WITHOUT_BRC_PRELOAD
    SoftmaxFlashV2<T, true, true, false, false, IFA_SOFTMAX_FLASHV2_CFG_WITHOUT_BRC>(
        mmResUb, softmaxSumUb[outIdx * SOFTMAX_SUM_BUF_SIZE + baseOffset],
        softmaxMaxUb[outIdx * SOFTMAX_MAX_BUF_SIZE + baseOffset], mmResUb,
            // ↑ 换行续接 ↓
            softmaxExpUb[outIdx * SOFTMAX_EXP_BUF_SIZE + baseOffset],
        inSumTensor, inMaxTensor, softmaxTmpUb, newTiling, srcShape);
#else
    SoftmaxFlashV2<T, true, true, false, false, IFA_SOFTMAX_FLASHV2_CFG>(
        mmResUb, softmaxSumUb[outIdx * SOFTMAX_SUM_BUF_SIZE + baseOffset],
        softmaxMaxUb[outIdx * SOFTMAX_MAX_BUF_SIZE + baseOffset], mmResUb,
            // ↑ 换行续接 ↓
            softmaxExpUb[outIdx * SOFTMAX_EXP_BUF_SIZE + baseOffset],
        inSumTensor, inMaxTensor, softmaxTmpUb, newTiling, srcShape);
#endif
}
```

十二、ComputeMm2函数 (行3546-3776)

功能说明: Softmax×V矩阵乘法完整流程

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::ComputeMm2(ExtraInfo&
    // ↑ 换行续接 ↓
```

```
    info) {
    constexpr uint32_t mCopyRowCount = P_LOAD_TO_L1_ROW_NUM;
    uint32_t mActRowCount = msdIterNum * info.mSize;
    uint32_t mCopyTimes = (mActRowCount + mCopyRowCount - 1) / mCopyRowCount;
    uint32_t mTailCopyRowCount = mActRowCount - (mCopyTimes - 1) * mCopyRowCount;

    constexpr uint32_t kCopyRowCount = KV_LOAD_TO_L1_ROW_NUM;
    uint32_t kCopyTimes = (info.actualSingleProcessSInnerSize + kCopyRowCount
        // ↑ 换行续接 ↓
        - 1) / kCopyRowCount;
    uint32_t kTailCopyRowCount = info.actualSingleProcessSInnerSize - (kCopyTimes
        // ↑ 换行续接 ↓
        - 1) * kCopyRowCount;
    uint32_t kTailCopyRowCountAlign = info.actualSingleProcessSInnerSizeAlign - (
        // ↑ 换行续接 ↓
        kCopyTimes - 1) * kCopyRowCount;

    for (uint32_t mCopyIdx = 0, mActCopyRowCount = mCopyRowCount; mCopyIdx
        // ↑ 换行续接 ↓
        < mCopyTimes; mCopyIdx++) {
        if (mCopyIdx + 1 == mCopyTimes) {
            mActCopyRowCount = mTailCopyRowCount;
        }
        LocalTensor<L0C_T> cL0Tensor = cL0TensorPingPong[(cL0BufIter % 2)
            // ↑ 换行续接 ↓
            * L0C_PP_SIZE / sizeof(L0C_T)];
        WaitFlag<HardEvent::FIX_M>(L0C_EVENT0 + (cL0BufIter % 2));
        for (uint32_t kCopyIdx = 0, kActCopyRowCount = kCopyRowCount, kActCopyRowCountAlign = kCopyRowCount;
            // ↑ 换行续接 ↓
            kCopyIdx < kCopyTimes; kCopyIdx++) {
            if (kCopyIdx + 1 == kCopyTimes) {
                kActCopyRowCount = kTailCopyRowCount;
                kActCopyRowCountAlign = kTailCopyRowCountAlign;
            }
            kpL1BufIter++;
            LocalTensor<KV_T> aL1Tensor = kpL1Buffers[(kpL1BufIter % 3)
                // ↑ 换行续接 ↓
                * L1KP_BLOCK_SIZE / sizeof(KV_T)];
            WaitFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT0 + (kpL1BufIter % 3));
            CopyInMm2AToL1(aL1Tensor, info, mCopyIdx, mCopyRowCount, mActCopyRowCount,
                // ↑ 换行续接 ↓
                kCopyIdx, kCopyRowCount, kActCopyRowCountAlign);
            SetFlag<HardEvent::MTE2_MTE1>(L1KP_EVENT0 + (kpL1BufIter % 3));
            WaitFlag<HardEvent::MTE2_MTE1>(L1KP_EVENT0 + (kpL1BufIter % 3));
            LocalTensor<KV_T> bL1Tensor = vL1Buffers[(vL1BufIter % 4) *
                // ↑ 换行续接 ↓
                L1V_BLOCK_SIZE / sizeof(KV_T)];
            uint32_t kb = 0;
            if (mCopyIdx == 0) {
                vL1BufIter++;
                bL1Tensor = vL1Buffers[(vL1BufIter % 4) * L1V_BLOCK_SIZE / sizeof(KV_T)];
                WaitFlag<HardEvent::MTE1_MTE2>(L1V_EVENT0 + (vL1BufIter % 4));
                if constexpr (PAGE_ATTENTION) {
                    uint64_t blockTableBaseOffset = info.bIdx * maxBlockNumPerBatch;
                    uint32_t curSeqIdx = info.s2BatchOffset + kCopyIdx * kCopyRowCount;
                    uint32_t copyFinishRowCnt = 0;
                    while (copyFinishRowCnt < kActCopyRowCount) {
                        uint64_t blockIdOffset = curSeqIdx / kvCacheBlockSize;
                            // ↑ 换行续接 ↓
                            // 获取blcok table上的索引
                        uint64_t reaminRowCnt = curSeqIdx % kvCacheBlockSize;
                            // ↑ 换行续接 ↓
                            // 获取在单个块上超出的行数
                        uint64_t idInBlockTable =
                            blockTableGm.GetValue(blockTableBaseOffset + blockIdOffset)
                                // ↑ 换行续接 ↓
                                ; // 从block table上的获取编号
                        // 计算可以拷贝行数
                        uint32_t copyRowCnt = kvCacheBlockSize - reaminRowCnt;
                        if (copyFinishRowCnt + copyRowCnt > kActCopyRowCount) {
                            copyRowCnt = kActCopyRowCount - copyFinishRowCnt;
                        }
                        uint64_t valueOffset = idInBlockTable * kvCacheBlockSize
                            // ↑ 换行续接 ↓
                            * headDim * kvHeadNum;
                        if constexpr (KV_LAYOUT_T == LAYOUT::NZ) {
                            uint32_t blockElementCnt = 32 / sizeof(KV_T);
                            if constexpr (KVINT4) {
                                blockElementCnt = 64;
```

```
                    }
                    valueOffset += (uint64_t)(info.n2Idx * headDim * kvCacheBlockSize)
                        // ↑ 换行续接 ↓
                        + reaminRowCnt * blockElementCnt;
                } else {
                    if constexpr (KV_LAYOUT_T == LAYOUT::BSH ||
                        // ↑ 换行续接 ↓
                        KV_LAYOUT_T == LAYOUT::BSND) {
                        valueOffset += (uint64_t)(info.n2Idx * headDim)
                            // ↑ 换行续接 ↓
                            + reaminRowCnt * headDim * kvHeadNum;
                    } else {
                        valueOffset += (uint64_t)(info.n2Idx * headDim * kvCacheBlockSize)
                            // ↑ 换行续接 ↓
                            + reaminRowCnt * headDim;
                    }
                }
                CopyInMm2BToL1ForPA(bL1Tensor, valueOffset, kActCopyRowCount,
                    // ↑ 换行续接 ↓
                    copyFinishRowCnt, copyRowCnt);

                // 更新循环变量
                copyFinishRowCnt += copyRowCnt;
                curSeqIdx += copyRowCnt;
            }
        } else {
            CopyInMm2BToL1(bL1Tensor, info, kCopyIdx, kCopyRowCount,
                // ↑ 换行续接 ↓
                kActCopyRowCount);
        }
        SetFlag<HardEvent::MTE2_MTE1>(L1V_EVENT0 + (vL1BufIter % 4));
        WaitFlag<HardEvent::MTE2_MTE1>(L1V_EVENT0 + (vL1BufIter % 4));
        kb = vL1BufIter;
    } else {
        kb = vL1BufIter - (kCopyTimes - kCopyIdx - 1);
        bL1Tensor = vL1Buffers[(kb % 4) * L1V_BLOCK_SIZE / sizeof(KV_T)];
    }
    constexpr uint32_t baseK = 128 / sizeof(KV_T);
    uint32_t kLoopTimes = (kActCopyRowCountAlign + baseK - 1) / baseK;
    uint32_t kTailAlign = kActCopyRowCountAlign - (kLoopTimes - 1) * baseK;
    uint32_t kTail = kActCopyRowCount - (kLoopTimes - 1) * baseK;
    for (uint32_t i = 0, actualBaseKAlign = baseK, actualBaseK = baseK;
        // ↑ 换行续接 ↓
        i < kLoopTimes; i++) {
        if (i + 1 == kLoopTimes) {
            actualBaseKAlign = kTailAlign;
            actualBaseK = kTail;
        }

        LocalTensor<KV_T> aL0Tensor = aL0TensorPingPong[(aL0BufIter % 2)
            // ↑ 换行续接 ↓
            * L0A_PP_SIZE / sizeof(KV_T)];
        WaitFlag<HardEvent::M_MTE1>(L0A_EVENT0 + (aL0BufIter % 2));
        LocalTensor<KV_T> bL0Tensor = bL0TensorPingPong[(bL0BufIter % 2)
            // ↑ 换行续接 ↓
            * L0B_PP_SIZE / sizeof(KV_T)];
        WaitFlag<HardEvent::M_MTE1>(L0B_EVENT0 + (bL0BufIter % 2));

        LocalTensor<KV_T> curAL1Tensor = aL1Tensor[16 * baseK * i];

        uint32_t mmRowCount = mActCopyRowCount;
        uint32_t copyStrideL0 = 16 * actualBaseKAlign;
        uint32_t copyStrideL1 = 16 * kActCopyRowCountAlign;
        uint32_t copyIterNum = (mmRowCount + 15) / 16;
        for(int i = 0; i < copyIterNum; i++){
            LoadDataMm2A(aL0Tensor[i * copyStrideL0], curAL1Tensor[i
                // ↑ 换行续接 ↓
                * copyStrideL1], actualBaseKAlign);
        }

        SetFlag<HardEvent::MTE1_M>(L0A_EVENT0 + (aL0BufIter % 2));
        WaitFlag<HardEvent::MTE1_M>(L0A_EVENT0 + (aL0BufIter % 2));

        uint32_t blockElementCnt = 32 / sizeof(KV_T);
        if constexpr (KVINT4) {
            blockElementCnt = 64;
        }
        LoadData2dTransposeParams loadData2DTransposeParamsForB;
        loadData2DTransposeParamsForB.startIndex = 0;
```

```
                loadData2DTransposeParamsForB.srcStride = 1;
                loadData2DTransposeParamsForB.dstFracGap = 0;
#ifdef L1_LAYOUT_zN
                loadData2DTransposeParamsForB.repeatTimes = actualBaseKAlign
                    // ↑ 换行续接 ↓
                    / blockElementCnt;
                loadData2DTransposeParamsForB.dstGap = headDimAlign / 16 - 1;
                uint32_t loadLoopTimes = headDimAlign / blockElementCnt;
                for(uint32_t j = 0; j < loadLoopTimes; j++){
                    LoadDataWithTranspose(bL0Tensor[j * blockElementCnt
                        // ↑ 换行续接 ↓
                        * blockElementCnt],
                        bL1Tensor[(i * baseK + j * kActCopyRowCountAlign)
                            // ↑ 换行续接 ↓
                            * blockElementCnt], loadData2DTransposeParamsForB);
                }
#else
                loadData2DTransposeParamsForB.repeatTimes = (actualBaseKAlign / blockElementCnt)
                    // ↑ 换行续接 ↓
                    * (headDimAlign / blockElementCnt);
                loadData2DTransposeParamsForB.dstGap = blockElementCnt / 16 - 1;
                uint32_t l1BaseOffset = baseK * headDimAlign * i;
                LoadDataWithTranspose(bL0Tensor, bL1Tensor[l1BaseOffset],
                    // ↑ 换行续接 ↓
                    loadData2DTransposeParamsForB);
#endif
                SetFlag<HardEvent::MTE1_M>(L0B_EVENT0 + (bL0BufIter % 2));
                WaitFlag<HardEvent::MTE1_M>(L0B_EVENT0 + (bL0BufIter % 2));

                MmadParams mmadParams;
                mmadParams.m = mActCopyRowCount;
                if (mmadParams.m == 1) {  // m等于1会默认开GEMV模式，且不可关闭GEMV，所以规避当作矩阵计算
                    mmadParams.m = 16;
                }
                mmadParams.n = 128;
                mmadParams.k = actualBaseK; // 无效数据不参与计算
                mmadParams.cmatrixInitVal = (kCopyIdx == 0) && (i == 0);
                mmadParams.cmatrixSource = false;
                Mmad(cL0Tensor, aL0Tensor, bL0Tensor, mmadParams);
                PipeBarrier<PIPE_M>();

                SetFlag<HardEvent::M_MTE1>(L0A_EVENT0 + (aL0BufIter % 2));
                aL0BufIter++;
                SetFlag<HardEvent::M_MTE1>(L0B_EVENT0 + (bL0BufIter % 2));
                bL0BufIter++;
            }

            SetFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT0 + (kpL1BufIter % 3));
            if ((mCopyIdx + 1) == mCopyTimes) {
                SetFlag<HardEvent::MTE1_MTE2>(L1V_EVENT0 + (kb % 4));
            }
        }
        SetFlag<HardEvent::M_FIX>(L0C_EVENT0 + (cL0BufIter % 2));
        WaitFlag<HardEvent::M_FIX>(L0C_EVENT0 + (cL0BufIter % 2));
        if (mCopyTimes == 1) {
            for (uint32_t mIter = 0; mIter < msdIterNum; mIter++) {
                float tmp = quantScaleC2O1;
                if (mIter == 1) {
                    tmp = quantScaleC2O2;
                }
                FixpipeParamsV220 fixParams;
                fixParams.nSize = 128;
                fixParams.mSize = mActCopyRowCount / msdIterNum; // 有效数据不足16行，只需要输出部分行即可
                fixParams.srcStride = ((msdIterNum * fixParams.mSize + 15) / 16) * 16;
                fixParams.dstStride = 128;
                fixParams.ndNum = 1;
                fixParams.quantPre = QuantMode_t::DEQF16;
                fixParams.deqScalar = static_cast<uint64_t>(*reinterpret_cast<int32_t
                    // ↑ 换行续接 ↓
                    *>(&tmp));
                if (mIter == 1) {
                    SetAtomicAdd<half>();
                }
                Fixpipe(mm2ResGm[(info.loop % (PRE_LOAD_NUM_DD)) * bmm2ResUbSize],
                    // ↑ 换行续接 ↓
                    cL0Tensor[mIter * fixParams.mSize * 16], fixParams);
                if (mIter == 1) {
                    SetAtomicNone();
```

```
                }
                PipeBarrier<PIPE_FIX>();
            }
        } else {
            if (mTailCopyRowCount != mCopyRowCount && mCopyIdx == 0) {
                float tmp = quantScaleC2O1;
                FixpipeParamsV220 fixParams;
                fixParams.nSize = 128;
252             fixParams.mSize = info.mSize; // 有效数据不足16行，只需要输出部分行即可
253             fixParams.srcStride = ((mActCopyRowCount + 15) / 16) * 16;
254             fixParams.dstStride = 128;
255             fixParams.ndNum = 1;
256             fixParams.quantPre = QuantMode_t::DEQF16;
257             fixParams.deqScalar = static_cast<uint64_t>(*reinterpret_cast<int32_t
258                 // ↑ 换行续接 ↓
259                 *>(&tmp));
260             Fixpipe(mm2ResGm[(info.loop % (PRE_LOAD_NUM_DD)) * bmm2ResUbSize],
261                 // ↑ 换行续接 ↓
262                 cL0Tensor, fixParams);
263             PipeBarrier<PIPE_FIX>();
264             tmp = quantScaleC2O2;
265             fixParams.mSize = mActCopyRowCount - info.mSize;
266             fixParams.deqScalar = static_cast<uint64_t>(*reinterpret_cast<int32_t
267                 // ↑ 换行续接 ↓
268                 *>(&tmp));
269             SetAtomicAdd<half>();
270             Fixpipe(mm2ResGm[(info.loop % (PRE_LOAD_NUM_DD)) * bmm2ResUbSize],
271                 // ↑ 换行续接 ↓
272                 cL0Tensor[info.mSize * 16], fixParams);
273             SetAtomicNone();
274             PipeBarrier<PIPE_FIX>();
275         } else if (mCopyIdx == 0) {
276             float tmp = quantScaleC2O1;
277             FixpipeParamsV220 fixParams;
278             fixParams.nSize = 128;
279             fixParams.mSize = mActCopyRowCount; // 有效数据不足16行，只需要输出部分行即可
280             fixParams.srcStride = ((mActCopyRowCount + 15) / 16) * 16;
281             fixParams.dstStride = 128;
282             fixParams.ndNum = 1;
283             fixParams.quantPre = QuantMode_t::DEQF16;
284             fixParams.deqScalar = static_cast<uint64_t>(*reinterpret_cast<int32_t
285                 // ↑ 换行续接 ↓
286                 *>(&tmp));
287             Fixpipe(mm2ResGm[(info.loop % (PRE_LOAD_NUM_DD)) * bmm2ResUbSize],
288                 // ↑ 换行续接 ↓
289                 cL0Tensor, fixParams);
290             PipeBarrier<PIPE_FIX>();
291         } else {
292             float tmp = quantScaleC2O2;
293             FixpipeParamsV220 fixParams;
294             fixParams.nSize = 128;
295             fixParams.mSize = mActCopyRowCount; // 有效数据不足16行，只需要输出部分行即可
296             fixParams.srcStride = ((mActCopyRowCount + 15) / 16) * 16;
297             fixParams.dstStride = 128;
298             fixParams.ndNum = 1;
299             fixParams.quantPre = QuantMode_t::DEQF16;
300             fixParams.deqScalar = static_cast<uint64_t>(*reinterpret_cast<int32_t
301                 // ↑ 换行续接 ↓
302                 *>(&tmp));
303             SetAtomicAdd<half>();
304             Fixpipe(mm2ResGm[(info.loop % (PRE_LOAD_NUM_DD)) * bmm2ResUbSize + (
305                 // ↑ 换行续接 ↓
306                 mCopyIdx * mCopyRowCount - info.mSize) * headDimAlign], cL0Tensor, fixParams);
307             SetAtomicNone();
308             PipeBarrier<PIPE_FIX>();
309         }
        }
        SetFlag<HardEvent::FIX_M>(L0C_EVENT0 + (cL0BufIter % 2));
        cL0BufIter++;
    }
}
```

十三、ProcessVec2Inner函数 (行2619-2640)

功能说明: MM2结果处理（反量化、累加）

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::ProcessVec2Inner(ExtraInfo&
    // ↑ 换行续接 ↓
    info)
{
    uint32_t mSplitSize = BASE_BLOCK_MAX_ELEMENT_NUM / headDimAlign;
    if (mSplitSize > mSizeVector) {
        mSplitSize = mSizeVector;
    }
    uint32_t loopCount = (mSizeVector + mSplitSize - 1) / mSplitSize;
    uint32_t tailSplitSize = mSizeVector - (loopCount - 1) * mSplitSize;

    for (uint32_t i = 0, dealSize = mSplitSize; i < loopCount; i++) {
        if (i == (loopCount - 1)) {
            dealSize = tailSplitSize;
        }
        if constexpr (ANTIQUANT) {
            if constexpr (ANTIQUANT_PER_CHANNEL) {
                DealAntiqBmm2ResBaseBlock(info, i * mSplitSize, dealSize,
                    // ↑ 换行续接 ↓
                    headDimAlign, headDim);
            } else if constexpr (ANTIQUANT_PER_TOKEN) {
                DealAntiqBmm2ResBaseBlockPerToken(info, i * mSplitSize, dealSize,
                    // ↑ 换行续接 ↓
                    headDimAlign, headDim);
            }
        }
    }
}
```

十四、Process主流程函数 (行3779-3954)

功能说明: Kernel主入口：Batch/Head/S循环

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::Process()
{

    if ASCEND_IS_AIV {
        SetFlag<AscendC::HardEvent::V_MTE2>(SYNC_INPUT_BUF1_FLAG);
        SetFlag<AscendC::HardEvent::V_MTE2>(SYNC_INPUT_BUF2_FLAG);
        SetFlag<AscendC::HardEvent::MTE3_V>(SYNC_OUTPUT_BUF1_FLAG);
        SetFlag<AscendC::HardEvent::MTE3_V>(SYNC_OUTPUT_BUF2_FLAG);
    }
    if (aiCoreIdx < usedCoreNum) {
        if ASCEND_IS_AIV {
#ifdef IFA_SOFTMAX_WITHOUT_BRC_PRELOAD
            Duplicate(softmaxMaxDefaultUb, SOFTMAX_MIN_NUM, SOFTMAX_MAX_BUF_SIZE);
            Duplicate(softmaxSumDefaultUb, FLOAT_ZERO, SOFTMAX_SUM_BUF_SIZE);
#else
            Duplicate(softmaxMaxDefaultUb, SOFTMAX_MIN_NUM, BUFFER_SIZE_BYTE_512B
                // ↑ 换行续接 ↓
                / sizeof(T));
            Duplicate(softmaxSumDefaultUb, FLOAT_ZERO, BUFFER_SIZE_BYTE_512B / sizeof(T));
#endif
        } else {
            SetFlag<HardEvent::MTE1_MTE2>(L1Q_EVENT0);
            SetFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT0);
            SetFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT1);
            SetFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT2);
            SetFlag<HardEvent::MTE1_MTE2>(L1V_EVENT0);
            SetFlag<HardEvent::MTE1_MTE2>(L1V_EVENT1);
            SetFlag<HardEvent::MTE1_MTE2>(L1V_EVENT2);
            SetFlag<HardEvent::MTE1_MTE2>(L1V_EVENT3);

            SetFlag<HardEvent::M_MTE1>(L0A_EVENT0);
            SetFlag<HardEvent::M_MTE1>(L0A_EVENT1);
            SetFlag<HardEvent::M_MTE1>(L0B_EVENT0);
            SetFlag<HardEvent::M_MTE1>(L0B_EVENT1);

            SetFlag<HardEvent::FIX_M>(L0C_EVENT0);
            SetFlag<HardEvent::FIX_M>(L0C_EVENT1);
        }

```

```
    ExtraInfo extraInfo[EXTRA_NUM];
    TaskContext taskContext[TASK_NUM];
    uint32_t gloop = 0;
    uint32_t gtasks = 0; //gtasks记录全局已处理了多少个Task
    uint32_t gtasksC1V1 = 0;
    uint32_t gtasksC2V2 = 0;
    uint32_t tasks = 0; //记录每组Task数量（预加载task的数量）

    if constexpr (!FLASH_DECODE) {
        for (uint32_t bn2gIdx = bn2LoopTimes; bn2gIdx > 0; bn2gIdx--) {
            GetBN2Gid(bn2gIdx - 1);
            GetActualSeqLen(bIdx, s1Idx);
            if (curActualSeqLen == 0) {
                DealActSeqLenIsZero(bIdx, n2Idx);
            }
            if (curActualSeqLen != 0 && actS1Size != 0) {
                break;
            }
            bn2LoopTimes--;
        }
    }

    for (uint32_t bn2gIdx = 0; bn2gIdx < bn2LoopTimes; bn2gIdx++) {
        GetBN2Gid(bn2gIdx);
        GetActualSeqLen(bIdx, s1Idx);
        UpdateInnerLoopCond();
        if (curActSeqLenIsZero) {
            DealActSeqLenIsZero(bIdx, n2Idx);
            continue;
        }
        if (actS1Size == 0) {
            continue;
        }
        uint32_t curS2LoopTimes = (bn2gIdx == bn2LoopTimes - 1) ? sInnerLoopTimes
            // ↑ 换行续接 ↓
            + 2 : sInnerLoopTimes;
        for (uint32_t sInnerLoopIdx = 0; sInnerLoopIdx < curS2LoopTimes;
            // ↑ 换行续接 ↓
            sInnerLoopIdx++) {
            if (sInnerLoopIdx < sInnerLoopTimes) {
                TaskContext &ctx = taskContext[tasks++];
                ctx.bidx = bIdx;
                ctx.nidx = n2Idx;
                ctx.s2idx = sInnerLoopIdx;
                ctx.s2loops = sInnerLoopTimes;
                ctx.gidx = gIdx;
                ctx.s2SizeTail = singleProcessSInnerSizeTail;
                ctx.s1Size = actS1Size;
                ctx.s2Size = curActualSeqLen;
                ctx.s1idx = s1Idx;
                ctx.actSeqLenQ = curActualSeqLenQ;
                bool isLast = (bn2gIdx == bn2LoopTimes - 1) && (sInnerLoopIdx
                    // ↑ 换行续接 ↓
                    >= sInnerLoopTimes - 1);
                if (tasks < TASK_NUM && !isLast) {
                    continue;
                }
                //遍历Task任务
                for (uint32_t i = 0; i < tasks; i++) {
                    uint32_t loop = gtasks;
                    CalcParams(loop, extraInfo[loop % EXTRA_NUM], taskContext[i]);

                    if ASCEND_IS_AIV {
                        if constexpr (ANTIQUANT) {
                            QueryPreProcessL(extraInfo[loop % EXTRA_NUM]);
                            CrossCoreSetFlag<SYNC_MODE2, PIPE_MTE3>(SYNC_V0_C1_FLAG);
                        }
                    }
                    gtasks++;
                }
            }

            for (uint32_t i = 0; i < TASK_NUM; i++) {
                uint32_t loop = gtasksC1V1;
                if (gloop > 0 && gtasksC1V1 < gtasks) {
                    if ASCEND_IS_AIC {
                        if constexpr (ANTIQUANT) {
                            CrossCoreWaitFlag(SYNC_V0_C1_FLAG);
                        }
```

```
                        ComputeMm1(extraInfo[loop % EXTRA_NUM]);
                        CrossCoreSetFlag<SYNC_MODE2, PIPE_FIX>(SYNC_C1_V1_FLAG);
                    }
                    if ASCEND_IS_AIV {
                        CrossCoreWaitFlag(SYNC_C1_V1_FLAG);
                        ProcessVec1L(extraInfo[loop % EXTRA_NUM]);
                        CrossCoreSetFlag<SYNC_MODE2, PIPE_MTE3>(SYNC_V1_C2_FLAG);
                    }
                    gtasksC1V1++;
                }
            }

            for (uint32_t i = 0; i < TASK_NUM; i++) {
                uint32_t loop = gtasksC2V2;
                if (gloop > 1 && gtasksC2V2 < gtasks) {
                    if ASCEND_IS_AIC {
                        CrossCoreWaitFlag(SYNC_V1_C2_FLAG);
                        ComputeMm2(extraInfo[loop % EXTRA_NUM]);
                        CrossCoreSetFlag<SYNC_MODE2, PIPE_FIX>(SYNC_C2_V2_FLAG);
                    }
                    if ASCEND_IS_AIV {
                        CrossCoreWaitFlag(SYNC_C2_V2_FLAG);
                        ProcessVec2L(extraInfo[loop % EXTRA_NUM]);
                    }
                    gtasksC2V2++;
                }
            }
            tasks = 0;
            gloop++;
        }
    }

    if ASCEND_IS_AIC {
        WaitFlag<HardEvent::MTE1_MTE2>(L1Q_EVENT0);
        WaitFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT0);
        WaitFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT1);
        WaitFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT2);
        WaitFlag<HardEvent::MTE1_MTE2>(L1V_EVENT0);
        WaitFlag<HardEvent::MTE1_MTE2>(L1V_EVENT1);
        WaitFlag<HardEvent::MTE1_MTE2>(L1V_EVENT2);
        WaitFlag<HardEvent::MTE1_MTE2>(L1V_EVENT3);

        WaitFlag<HardEvent::M_MTE1>(L0A_EVENT0);
        WaitFlag<HardEvent::M_MTE1>(L0A_EVENT1);
        WaitFlag<HardEvent::M_MTE1>(L0B_EVENT0);
        WaitFlag<HardEvent::M_MTE1>(L0B_EVENT1);

        WaitFlag<HardEvent::FIX_M>(L0C_EVENT0);
        WaitFlag<HardEvent::FIX_M>(L0C_EVENT1);
    }
}

if constexpr (FLASH_DECODE) {
    // 多核同步
    SyncAll();
    if ASCEND_IS_AIV {
        FlashDecodeCompute();
    }
}
if ASCEND_IS_AIV {
    WaitFlag<AscendC::HardEvent::V_MTE2>(SYNC_INPUT_BUF1_FLAG);
    WaitFlag<AscendC::HardEvent::V_MTE2>(SYNC_INPUT_BUF2_FLAG);
    WaitFlag<AscendC::HardEvent::MTE3_V>(SYNC_OUTPUT_BUF1_FLAG);
    WaitFlag<AscendC::HardEvent::MTE3_V>(SYNC_OUTPUT_BUF2_FLAG);
}
}
```

十五、CopyInMm1AToL1函数 (行2785-2816)

功能说明: Query数据从GM搬运到L1

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::CopyInMm1AToL1(
    // ↑ 换行续接 ↓
    LocalTensor<KV_T>& aL1Tensor, ExtraInfo& info) {
```

```
    uint32_t mmRowCount = msdIterNum * info.mSize;
    uint32_t copyStride = 16 * headDimAlign;
    uint32_t copyIterNum = (mmRowCount + 15) / 16;
    for(int i = 0; i < copyIterNum; i++) {
        Nd2NzParams mm1Nd2NzParamsForA;
        mm1Nd2NzParamsForA.ndNum = 1; // ND矩阵的个数
        if(i == copyIterNum - 1) {
            mm1Nd2NzParamsForA.nValue = msdIterNum * info.mSize - i * 16;
        }
        else {
            mm1Nd2NzParamsForA.nValue = 16; // 单个ND矩阵的行数，单位为元素个数  16
        }
        if constexpr (KVINT4) {
            mm1Nd2NzParamsForA.dValue = headDimAlign / 2;
            mm1Nd2NzParamsForA.srcDValue = headDimAlign / 2; // 同一个ND矩阵中相邻行起始地址之间的偏移，
                // ↑ 换行续接 ↓
                单位为元素个数
        } else {
            mm1Nd2NzParamsForA.dValue = headDimAlign; // 单个ND矩阵的列数，单位为元素个数   128
            mm1Nd2NzParamsForA.srcDValue = headDimAlign; // 同一个ND矩阵中相邻行起始地址之间的偏移，
                // ↑ 换行续接 ↓
                单位为元素个数  128
        }
        mm1Nd2NzParamsForA.srcNdMatrixStride = 0; // 相邻ND矩阵起始地址之间的偏移，单位为元素个数
        mm1Nd2NzParamsForA.dstNzC0Stride = 16; // 转换为NZ矩阵后，相邻Block起始地址之间的偏移，
            // ↑ 换行续接 ↓
            单位为Block个数  16
        mm1Nd2NzParamsForA.dstNzNStride = 1; // 转换为NZ矩阵后，ND中之前相邻两行在NZ矩阵中起始地址之间的偏移
        mm1Nd2NzParamsForA.dstNzMatrixStride = 0; // 两个NZ矩阵，起始地址之间的偏移
        if constexpr (ANTIQUANT) {
            DataCopy(aL1Tensor[i * copyStride], queryPreProcessResGm[(info.bn2IdxInCurCore % (
                // ↑ 换行续接 ↓
                PRE_LOAD_NUM_DD)) * bmm2ResUbSize + i * copyStride], mm1Nd2NzParamsForA);
        } else {
            DataCopy(aL1Tensor, queryGm[info.tensorAOffset], mm1Nd2NzParamsForA);
        }
    }
    auto b = queryPreProcessResGm[(info.bn2IdxInCurCore % (PRE_LOAD_NUM_DD))
        // ↑ 换行续接 ↓
        * bmm2ResUbSize];
}
```

十六、CopyInMm1BToL1函数 (行2820-2904)

功能说明: Key数据从GM搬运到L1

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::CopyInMm1BToL1(
    LocalTensor<KV_T>& bL1Tensor, ExtraInfo& info, uint32_t nCopyIdx, uint32_t
        // ↑ 换行续接 ↓
        nCopyRowCount,
    uint32_t nActCopyRowCount, uint32_t nActCopyRowCountAlign)
{
    uint64_t step = headDim;
    if constexpr (KV_LAYOUT_T == LAYOUT::BSH || KV_LAYOUT_T == LAYOUT::BSND) {
        step = headDim * kvHeadNum;
    }
    uint64_t keyGmBaseOffset = info.tensorBOffset + nCopyIdx * nCopyRowCount * step;

#ifdef L1_LAYOUT_zN
    Nd2NzParams mm1Nd2NzParamsForB;
    mm1Nd2NzParamsForB.ndNum = 1;
    mm1Nd2NzParamsForB.nValue = nActCopyRowCount;
    if constexpr (KVINT4) {
        mm1Nd2NzParamsForB.dValue = headDim / 2;
        mm1Nd2NzParamsForB.srcDValue = step / 2;
    } else {
        mm1Nd2NzParamsForB.dValue = headDim;
        mm1Nd2NzParamsForB.srcDValue = step;
    }
    mm1Nd2NzParamsForB.dstNzC0Stride = nActCopyRowCountAlign;
    mm1Nd2NzParamsForB.dstNzNStride = 1;
    mm1Nd2NzParamsForB.srcNdMatrixStride = 0;
    mm1Nd2NzParamsForB.dstNzMatrixStride = 0;
    DataCopy(bL1Tensor, keyGm[keyGmBaseOffset], mm1Nd2NzParamsForB);
```

```
#else
    uint32_t baseN = 256 / sizeof(KV_T);
    uint32_t ndNum_tmp = nActCopyRowCount / baseN;
    uint32_t nTail = nActCopyRowCount % baseN;
    uint32_t nTailAlign = nActCopyRowCountAlign - ndNum_tmp * baseN;

    Nd2NzParams mm1Nd2NzParamsForB;
    if (ndNum_tmp != 0){
        mm1Nd2NzParamsForB.nValue = baseN; // 单个ND矩阵的行数，单位为元素个数   256
        if constexpr (KVINT4) {
            mm1Nd2NzParamsForB.dValue = headDim / 2; // 单个ND矩阵的列数，单位为元素个数   128
            mm1Nd2NzParamsForB.srcDValue = step / 2; // 同一个ND矩阵中相邻行起始地址之间的偏移，
                // ↑ 换行续接 ↓
                单位为元素个数  // 128
        } else {
            mm1Nd2NzParamsForB.dValue = headDim; // 单个ND矩阵的列数，单位为元素个数   128
            mm1Nd2NzParamsForB.srcDValue = step; // 同一个ND矩阵中相邻行起始地址之间的偏移，单位为元素个数  // 128
        }
        mm1Nd2NzParamsForB.dstNzC0Stride = baseN;// 转换为NZ矩阵后，相邻Block起始地址之间的偏移，
            // ↑ 换行续接 ↓
            单位为Block个数   256
        mm1Nd2NzParamsForB.dstNzNStride = 1; // 转换为NZ矩阵后，ND中之前相邻两行在NZ矩阵中起始地址之间的偏移，
            // ↑ 换行续接 ↓
            单位为Block个数

        if ((KV_LAYOUT_T == LAYOUT::BSH || KV_LAYOUT_T == LAYOUT::BSND) && (
            // ↑ 换行续接 ↓
            baseN * step > 65535)) {
            mm1Nd2NzParamsForB.ndNum = 1;
            mm1Nd2NzParamsForB.srcNdMatrixStride = 0; // 相邻ND矩阵起始地址之间的偏移，单位为元素个数 256*128
            mm1Nd2NzParamsForB.dstNzMatrixStride = 0; // 两个NZ矩阵，起始地址之间的偏移，单位为元素个数 256*128
            for (uint32_t i = 0; i < ndNum_tmp; i++) {
                DataCopy(bL1Tensor[i * baseN * headDimAlign], keyGm[keyGmBaseOffset + i * baseN * step],
                    // ↑ 换行续接 ↓
                    mm1Nd2NzParamsForB);
                }
        } else {
            mm1Nd2NzParamsForB.ndNum = ndNum_tmp;
            if constexpr (KVINT4) {
                mm1Nd2NzParamsForB.srcNdMatrixStride = baseN * step / 2;
                    // ↑ 换行续接 ↓
                    // 相邻ND矩阵起始地址之间的偏移，单位为元素个数 256*128
                mm1Nd2NzParamsForB.dstNzMatrixStride = baseN * headDimAlign / 2;
                    // ↑ 换行续接 ↓
                    // 两个NZ矩阵，起始地址之间的偏移，单位为元素个数 256*128
            } else {
                mm1Nd2NzParamsForB.srcNdMatrixStride = baseN * step; //
                    // ↑ 换行续接 ↓
                    相邻ND矩阵起始地址之间的偏移，单位为元素个数 256*128
                mm1Nd2NzParamsForB.dstNzMatrixStride = baseN * headDimAlign;
                    // ↑ 换行续接 ↓
                    // 两个NZ矩阵，起始地址之间的偏移，单位为元素个数 256*128
            }
            DataCopy(bL1Tensor, keyGm[keyGmBaseOffset], mm1Nd2NzParamsForB);
        }
    }

    if (nTail != 0){
        mm1Nd2NzParamsForB.ndNum = 1;
        mm1Nd2NzParamsForB.nValue = nTail; // 单个ND矩阵的行数，单位为元素个数
actualSingleProcessSInnerSizeAlign为32对齐，nTail已经是32对齐
        if constexpr (KVINT4) {
            mm1Nd2NzParamsForB.dValue = headDim / 2; // 单个ND矩阵的列数，单位为元素个数   128
            mm1Nd2NzParamsForB.srcDValue = step / 2; // 同一个ND矩阵中相邻行起始地址之间的偏移，
                // ↑ 换行续接 ↓
                单位为元素个数  // 128
        } else {
            mm1Nd2NzParamsForB.dValue = headDim; // 单个ND矩阵的列数，单位为元素个数   128
            mm1Nd2NzParamsForB.srcDValue = step; // 同一个ND矩阵中相邻行起始地址之间的偏移，单位为元素个数  // 128
        }
        mm1Nd2NzParamsForB.srcNdMatrixStride = 0; // 相邻ND矩阵起始地址之间的偏移，单位为元素个数 0
        mm1Nd2NzParamsForB.dstNzC0Stride = nTailAlign;// 转换为NZ矩阵后，相邻Block起始地址之间的偏移，
            // ↑ 换行续接 ↓
            单位为Block个数
        mm1Nd2NzParamsForB.dstNzNStride = 1; // 转换为NZ矩阵后，ND中之前相邻两行在NZ矩阵中起始地址之间的偏移，
            // ↑ 换行续接 ↓
            单位为Block个数
        mm1Nd2NzParamsForB.dstNzMatrixStride = 0; // 两个NZ矩阵，起始地址之间的偏移，单位为元素个数  0

```

```
        DataCopy(bL1Tensor[ndNum_tmp * baseN * headDimAlign], keyGm[keyGmBaseOffset
            // ↑ 换行续接 ↓
            + ndNum_tmp * baseN * step],
                mm1Nd2NzParamsForB);  //需要调整偏移地址，bL1Tensor的偏移， keyGm的偏移
    }
#endif
}
```

十七、CopyInMm1BToL1ForPA函数 (行2907-3043)

功能说明: Page Attention三段式Key搬运

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::CopyInMm1BToL1ForPA(
    LocalTensor<KV_T>& bL1Tensor, uint64_t keyGmBaseOffset, uint32_t copyTotalRowCnt,
        // ↑ 换行续接 ↓
        uint32_t copyStartRowCnt, uint32_t nActCopyRowCount)
{
    uint32_t baseN = 256 / sizeof(KV_T);
    uint32_t blockElementCnt = 32 / sizeof(KV_T);
    if constexpr (KVINT4) {
        blockElementCnt = 64;
    }

    uint64_t step = headDim;
    if constexpr (KV_LAYOUT_T == LAYOUT::BSH || KV_LAYOUT_T == LAYOUT::BSND) {
        step = headDim * kvHeadNum;
    } else if constexpr (KV_LAYOUT_T == LAYOUT::NZ) {
        step = blockElementCnt;
    }

    uint32_t nHead = 0;
    uint32_t nTail = 0;
    uint32_t midNdNum = 0;
    uint32_t copyStartRowCntInBaseN = copyStartRowCnt % baseN;
    uint32_t copyStartRowCntInBaseNTail = 0;
    if (copyStartRowCntInBaseN + nActCopyRowCount <= baseN) {
        nHead = 0;
        midNdNum = 0;
        nTail = nActCopyRowCount;
        copyStartRowCntInBaseNTail = copyStartRowCntInBaseN;
    } else {
        if (copyStartRowCntInBaseN == 0) {
            nHead = 0;
        } else {
            nHead = baseN - copyStartRowCntInBaseN;
        }
        midNdNum = (nActCopyRowCount - nHead) / baseN;
        nTail = (nActCopyRowCount - nHead) % baseN;
        copyStartRowCntInBaseNTail = 0;
    }

    uint32_t dstNzC0StrideTail = baseN;
    if (copyTotalRowCnt % baseN != 0) {
        if ((copyStartRowCnt + nActCopyRowCount) > (copyTotalRowCnt / baseN * baseN)) {
            dstNzC0StrideTail = copyTotalRowCnt % baseN;
        }
    }

    Nd2NzParams mm1Nd2NzParamsForB;
    if (nHead != 0) {
        if constexpr (KV_LAYOUT_T == LAYOUT::NZ) {
            DataCopyParams intriParams;
            intriParams.blockLen = nHead;
            intriParams.blockCount = headDim / blockElementCnt;
            intriParams.dstStride = baseN - nHead;
            intriParams.srcStride = kvCacheBlockSize - nHead;

            uint32_t ndNumFinish = copyStartRowCnt / baseN;
            DataCopy(bL1Tensor[ndNumFinish * baseN * headDimAlign + copyStartRowCntInBaseN * (
                // ↑ 换行续接 ↓
                32 / sizeof(KV_T))], keyGm[keyGmBaseOffset], intriParams);
            copyTotalRowCnt -= nHead;
        } else {
            mm1Nd2NzParamsForB.ndNum = 1;
```

```
            mm1Nd2NzParamsForB.nValue = nHead; // 单个ND矩阵的行数，单位为元素个数
            mm1Nd2NzParamsForB.dValue = headDim; // 单个ND矩阵的列数，单位为元素个数
            mm1Nd2NzParamsForB.srcNdMatrixStride = 0; // 相邻ND矩阵起始地址之间的偏移，单位为元素个数
            mm1Nd2NzParamsForB.srcDValue = step; // 同一个ND矩阵中相邻行起始地址之间的偏移，单位为元素个数
            mm1Nd2NzParamsForB.dstNzC0Stride = baseN;// 转换为NZ矩阵后，相邻Block起始地址之间的偏移，
                // ↑ 换行续接 ↓
                单位为Block个数
            mm1Nd2NzParamsForB.dstNzNStride = 1; // 转换为NZ矩阵后，ND中之前相邻两行在NZ矩阵中起始地址之间的偏移，
                // ↑ 换行续接 ↓
                单位为Block个数
            mm1Nd2NzParamsForB.dstNzMatrixStride = 0; // 两个NZ矩阵，起始地址之间的偏移，单位为元素个数

            uint32_t ndNumFinish = copyStartRowCnt / baseN;
            DataCopy(bL1Tensor[ndNumFinish * baseN * headDimAlign + copyStartRowCntInBaseN * (
                // ↑ 换行续接 ↓
                32 / sizeof(KV_T))], keyGm[keyGmBaseOffset],
                    mm1Nd2NzParamsForB);
        }
    }

    if (midNdNum != 0) {
        if constexpr (KV_LAYOUT_T == LAYOUT::NZ) {
            DataCopyParams intriParams;
            intriParams.blockLen = baseN;
            intriParams.blockCount = headDim / blockElementCnt;
            intriParams.dstStride = 0;
            intriParams.srcStride = kvCacheBlockSize - baseN;

            int32_t ndNumFinish = (copyStartRowCnt + nHead) / baseN;
            for (uint32_t i = 0; i < midNdNum; i++) {
                DataCopy(bL1Tensor[(ndNumFinish + i) * baseN * headDimAlign],
                    // ↑ 换行续接 ↓
                    keyGm[keyGmBaseOffset + nHead * step + i * baseN * step], intriParams);
            }
            copyTotalRowCnt -= midNdNum * baseN;
        } else {
            mm1Nd2NzParamsForB.nValue = baseN; // 单个ND矩阵的行数，单位为元素个数   256
            mm1Nd2NzParamsForB.dValue = headDim; // 单个ND矩阵的列数，单位为元素个数   128
            mm1Nd2NzParamsForB.srcDValue = step; // 同一个ND矩阵中相邻行起始地址之间的偏移，单位为元素个数  // 128
            mm1Nd2NzParamsForB.dstNzC0Stride = baseN;// 转换为NZ矩阵后，相邻Block起始地址之间的偏移，
                // ↑ 换行续接 ↓
                单位为Block个数   256
            mm1Nd2NzParamsForB.dstNzNStride = 1; // 转换为NZ矩阵后，ND中之前相邻两行在NZ矩阵中起始地址之间的偏移，
                // ↑ 换行续接 ↓
                单位为Block个数

            int32_t ndNumFinish = (copyStartRowCnt + nHead) / baseN;
            if ((KV_LAYOUT_T == LAYOUT::BSH || KV_LAYOUT_T == LAYOUT::BSND)
                // ↑ 换行续接 ↓
                && (baseN * step > 65535)) {
                mm1Nd2NzParamsForB.ndNum = 1;
                mm1Nd2NzParamsForB.srcNdMatrixStride = 0; // 相邻ND矩阵起始地址之间的偏移，
                    // ↑ 换行续接 ↓
                    单位为元素个数 256*128
                mm1Nd2NzParamsForB.dstNzMatrixStride = 0; // 两个NZ矩阵，起始地址之间的偏移，
                    // ↑ 换行续接 ↓
                    单位为元素个数 256*128
                for (uint32_t i = 0; i < midNdNum; i++) {
                    DataCopy(bL1Tensor[(ndNumFinish + i) * baseN * headDimAlign],
                        // ↑ 换行续接 ↓
                        keyGm[keyGmBaseOffset + nHead * step + i * baseN * step], mm1Nd2NzParamsForB);
                }
            } else {
                mm1Nd2NzParamsForB.ndNum = midNdNum;
                mm1Nd2NzParamsForB.srcNdMatrixStride = baseN * step; //
                    // ↑ 换行续接 ↓
                    相邻ND矩阵起始地址之间的偏移，单位为元素个数 256*128
                mm1Nd2NzParamsForB.dstNzMatrixStride = baseN * headDimAlign;
                    // ↑ 换行续接 ↓
                    // 两个NZ矩阵，起始地址之间的偏移，单位为元素个数 256*128

                DataCopy(bL1Tensor[ndNumFinish * baseN * headDimAlign], keyGm[keyGmBaseOffset + nHead * step],
                    // ↑ 换行续接 ↓
                    mm1Nd2NzParamsForB);
            }
        }
    }

    if (nTail != 0) {

```

```
        if constexpr (KV_LAYOUT_T == LAYOUT::NZ) {
            DataCopyParams intriParams;
            intriParams.blockLen = nTail;
            intriParams.blockCount = headDim / blockElementCnt;
            intriParams.dstStride = dstNzC0StrideTail - nTail;
            intriParams.srcStride = kvCacheBlockSize - nTail;

            uint32_t ndNumFinish = (copyStartRowCnt + nHead) / baseN + midNdNum;
            DataCopy(bL1Tensor[ndNumFinish * baseN * headDimAlign + copyStartRowCntInBaseNTail * (
                // ↑ 换行续接 ↓
                32 / sizeof(KV_T))],
                keyGm[keyGmBaseOffset + (nHead + midNdNum * baseN) * step], intriParams);
        } else {
            mm1Nd2NzParamsForB.ndNum = 1;
            mm1Nd2NzParamsForB.nValue = nTail; // 单个ND矩阵的行数，单位为元素个数
actualSingleProcessSInnerSizeAlign为32对齐，nTail已经是32对齐
            mm1Nd2NzParamsForB.dValue = headDim; // 单个ND矩阵的列数，单位为元素个数   128
            mm1Nd2NzParamsForB.srcNdMatrixStride = 0; // 相邻ND矩阵起始地址之间的偏移，单位为元素个数 0
            mm1Nd2NzParamsForB.srcDValue = step; // 同一个ND矩阵中相邻行起始地址之间的偏移，单位为元素个数  // 128
            mm1Nd2NzParamsForB.dstNzC0Stride = dstNzC0StrideTail;// 转换为NZ矩阵后，相邻Block起始地址之间的偏移，
                // ↑ 换行续接 ↓
                单位为Block个数
            mm1Nd2NzParamsForB.dstNzNStride = 1; // 转换为NZ矩阵后，ND中之前相邻两行在NZ矩阵中起始地址之间的偏移，
                // ↑ 换行续接 ↓
                单位为Block个数
            mm1Nd2NzParamsForB.dstNzMatrixStride = 0; // 两个NZ矩阵，起始地址之间的偏移，单位为元素个数  0

            int32_t ndNumFinish = (copyStartRowCnt + nHead) / baseN + midNdNum;
            DataCopy(bL1Tensor[ndNumFinish * baseN * headDimAlign + copyStartRowCntInBaseNTail * (
                // ↑ 换行续接 ↓
                32 / sizeof(KV_T))], keyGm[keyGmBaseOffset + (nHead + midNdNum * baseN) * step],
                    mm1Nd2NzParamsForB);  //需要调整偏移地址，bL1Tensor的偏移， keyGm的偏移
        }
    }
}
```

十八、CopyInMm2AToL1函数 (行3302-3328)

功能说明: Softmax结果从Workspace搬运到L1

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::CopyInMm2AToL1(
    // ↑ 换行续接 ↓
    LocalTensor<KV_T>& aL1Tensor, ExtraInfo& info, uint32_t mCopyIdx, uint32_t mCopyRowCount, uint32_t mActCo
pyRowCount, uint32_t kCopyIdx, uint32_t kCopyRowCount, uint32_t kActCopyRowCount) {
    uint32_t mmRowCount = mActCopyRowCount;
    uint32_t copyStrideL1 = 16 * kActCopyRowCount;
    uint32_t copyStrideGm = 16 * info.actualSingleProcessSInnerSizeAlign;
    uint32_t copyIterNum = (mmRowCount + 15) / 16;
    for(int i = 0; i < copyIterNum; i++){
        Nd2NzParams mm1Nd2NzParamsForA;
        mm1Nd2NzParamsForA.ndNum = 1; // ND矩阵的个数
        if(i == copyIterNum - 1) {
            mm1Nd2NzParamsForA.nValue = mmRowCount - i * 16;
        }
        else {
            mm1Nd2NzParamsForA.nValue = 16; // 单个ND矩阵的行数，单位为元素个数  16
        }
        mm1Nd2NzParamsForA.dValue = kActCopyRowCount; // 单个ND矩阵的列数，单位为元素个数
        mm1Nd2NzParamsForA.srcNdMatrixStride = 0; // 相邻ND矩阵起始地址之间的偏移，单位为元素个数
        if constexpr (KVINT4) {
            mm1Nd2NzParamsForA.srcDValue = info.actualSingleProcessSInnerSizeAlign / 2;
        } else {
            mm1Nd2NzParamsForA.srcDValue = info.actualSingleProcessSInnerSizeAlign;
                // ↑ 换行续接 ↓
                // 同一个ND矩阵中相邻行起始地址之间的偏移，单位为元素个数
        }
        mm1Nd2NzParamsForA.dstNzC0Stride = 16; // 转换为NZ矩阵后，相邻Block起始地址之间的偏移，单位为Block个数
        mm1Nd2NzParamsForA.dstNzNStride = 1; // 转换为NZ矩阵后，ND中之前相邻两行在NZ矩阵中起始地址之间的偏移
        mm1Nd2NzParamsForA.dstNzMatrixStride = 0; // 两个NZ矩阵，起始地址之间的偏移
        DataCopy(aL1Tensor[i * copyStrideL1], vec1ResGm[(info.loop % (PRE_LOAD_NUM_DD)
            // ↑ 换行续接 ↓
            ) * mmResUbSize + (mCopyIdx * mCopyRowCount) * info.actualSingleProcessSInnerSizeAlign + kCopyIdx
* kCopyRowCount + i * copyStrideGm], mm1Nd2NzParamsForA);
```

```
    }
}
```

十九、CopyInMm2BToL1函数 (行3331-3402)

功能说明: Value数据从GM搬运到L1

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::CopyInMm2BToL1(
    // ↑ 换行续接 ↓
    LocalTensor<KV_T>& bL1Tensor, ExtraInfo& info, uint32_t kCopyIdx, uint32_t kCopyRowCount, uint32_t kActCo
pyRowCount) {
    uint64_t step = headDim;
    if constexpr (KV_LAYOUT_T == LAYOUT::BSH || KV_LAYOUT_T == LAYOUT::BSND) {
        step = headDim * kvHeadNum;
    }
    uint64_t valueGmBaseOffset = info.tensorBOffset + kCopyIdx * kCopyRowCount * step;

#ifdef L1_LAYOUT_zN
    Nd2NzParams mm1Nd2NzParamsForB;
    mm1Nd2NzParamsForB.ndNum = 1;
    mm1Nd2NzParamsForB.nValue = kActCopyRowCount;
    if constexpr (KVINT4) {
        mm1Nd2NzParamsForB.dValue = headDim / 2;
        mm1Nd2NzParamsForB.srcDValue = step / 2;
    } else {
        mm1Nd2NzParamsForB.dValue = headDim;
        mm1Nd2NzParamsForB.srcDValue = step;
    }
    mm1Nd2NzParamsForB.dstNzC0Stride = Align(kActCopyRowCount, 32U);
    mm1Nd2NzParamsForB.dstNzNStride = 1;
    mm1Nd2NzParamsForB.srcNdMatrixStride = 0;
    mm1Nd2NzParamsForB.dstNzMatrixStride = 0;
    DataCopy(bL1Tensor, valueGm[valueGmBaseOffset], mm1Nd2NzParamsForB);
#else
    uint32_t blockK = 32 / sizeof(KV_T); // 单个ND矩阵的行数
    if constexpr (KVINT4) {
        blockK = 64;
    }
    uint32_t kTail = kActCopyRowCount % blockK;
    uint32_t ndNum_tmp = kActCopyRowCount / blockK;

    Nd2NzParams mm1Nd2NzParamsForB;
    if constexpr (KVINT4) {
        mm1Nd2NzParamsForB.dValue = headDim / 2;
        mm1Nd2NzParamsForB.srcDValue = step / 2;
    } else {
        mm1Nd2NzParamsForB.dValue = headDim;
        mm1Nd2NzParamsForB.srcDValue = step; // 同一个ND矩阵中相邻行起始地址之间的偏移，单位为元素个数  // 128
    }
    mm1Nd2NzParamsForB.dstNzC0Stride = blockK;
    mm1Nd2NzParamsForB.dstNzNStride = 1;
    if (ndNum_tmp != 0){
        if ((KV_LAYOUT_T == LAYOUT::BSH || KV_LAYOUT_T == LAYOUT::BSND) && (
            // ↑ 换行续接 ↓
            blockK * step > 65535)) {
            mm1Nd2NzParamsForB.ndNum = 1;
            mm1Nd2NzParamsForB.nValue = blockK;
            mm1Nd2NzParamsForB.srcNdMatrixStride = 0;
            mm1Nd2NzParamsForB.dstNzMatrixStride = 0;
            for (uint32_t i = 0; i < ndNum_tmp; i++) {
                DataCopy(bL1Tensor[i * blockK * headDimAlign], valueGm[valueGmBaseOffset + i * blockK * step],
                    // ↑ 换行续接 ↓
                    mm1Nd2NzParamsForB);
            }
        } else {
            mm1Nd2NzParamsForB.ndNum = ndNum_tmp;
            mm1Nd2NzParamsForB.nValue = blockK;
            if constexpr (KVINT4) {
                mm1Nd2NzParamsForB.srcNdMatrixStride = blockK * step / 2;
                mm1Nd2NzParamsForB.dstNzMatrixStride = blockK * headDimAlign / 2;
            } else {
                mm1Nd2NzParamsForB.srcNdMatrixStride = blockK * step;
                mm1Nd2NzParamsForB.dstNzMatrixStride = blockK * headDimAlign;
```

```
            }
            DataCopy(bL1Tensor, valueGm[valueGmBaseOffset], mm1Nd2NzParamsForB);
        }
    }
    if (kTail != 0){
        mm1Nd2NzParamsForB.ndNum = 1;
        mm1Nd2NzParamsForB.nValue = kTail;
        mm1Nd2NzParamsForB.srcNdMatrixStride = 0;
        mm1Nd2NzParamsForB.dstNzMatrixStride = 0;
        DataCopy(bL1Tensor[ndNum_tmp * blockK * headDimAlign], valueGm[valueGmBaseOffset + ndNum_tmp * blockK
* step],
            // ↑ 换行续接 ↓
            mm1Nd2NzParamsForB);
    }
#endif
}
```

二十、CopyInMm2BToL1ForPA函数 (行3405-3524)

功能说明: Page Attention三段式Value搬运

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::CopyInMm2BToL1ForPA(
    LocalTensor<KV_T>& bL1Tensor, uint64_t valueGmBaseOffset, uint32_t copyTotalRowCnt,
        // ↑ 换行续接 ↓
        uint32_t copyStartRowCnt, uint32_t kActCopyRowCount)
{
    uint32_t blockK = 32 / sizeof(KV_T); // 单个ND矩阵的行数
    uint32_t blockElementCnt = 32 / sizeof(KV_T);
    if constexpr (KVINT4) {
        blockElementCnt = 64;
    }

    uint64_t step = headDim;
    if constexpr (KV_LAYOUT_T == LAYOUT::BSH || KV_LAYOUT_T == LAYOUT::BSND) {
        step = headDim * kvHeadNum;
    } else if constexpr (KV_LAYOUT_T == LAYOUT::NZ) {
        step = blockElementCnt;
    }

    uint32_t nHead = 0;
    uint32_t nTail = 0;
    uint32_t midNdNum = 0;
    uint32_t copyStartRowCntInABlockK = copyStartRowCnt % blockK;
    uint32_t copyStartRowCntInBaseNTail = 0;
    if (copyStartRowCntInABlockK + kActCopyRowCount <= blockK) {
        nHead = 0;
        midNdNum = 0;
        nTail = kActCopyRowCount;
        copyStartRowCntInBaseNTail = copyStartRowCntInABlockK;
    } else {
        if (copyStartRowCntInABlockK == 0) {
            nHead = 0;
        } else {
            nHead = blockK - copyStartRowCntInABlockK;
        }
        midNdNum = (kActCopyRowCount - nHead) / blockK;
        nTail = (kActCopyRowCount - nHead) % blockK;
        copyStartRowCntInBaseNTail = 0;
    }

    Nd2NzParams mm1Nd2NzParamsForB;
    mm1Nd2NzParamsForB.dValue = headDim;
    mm1Nd2NzParamsForB.srcDValue = step; // 同一个ND矩阵中相邻行起始地址之间的偏移，单位为元素个数
    mm1Nd2NzParamsForB.dstNzC0Stride = blockK;
    mm1Nd2NzParamsForB.dstNzNStride = 1;
    if (nHead != 0) {
        if constexpr (KV_LAYOUT_T == LAYOUT::NZ) {
            DataCopyParams intriParams;
            intriParams.blockLen = nHead;
            intriParams.blockCount = headDim / blockElementCnt;
            intriParams.dstStride = blockK - nHead;
            intriParams.srcStride = kvCacheBlockSize - nHead;

            uint32_t ndNumFinish = copyStartRowCnt / blockK;
```

```
            DataCopy(bL1Tensor[ndNumFinish * blockK * headDimAlign + copyStartRowCntInABlockK * (
                // ↑ 换行续接 ↓
                32 / sizeof(KV_T))], valueGm[valueGmBaseOffset], intriParams);
        } else {
            mm1Nd2NzParamsForB.ndNum = 1;
            mm1Nd2NzParamsForB.nValue = nHead; // 单个ND矩阵的行数，单位为元素个数
            mm1Nd2NzParamsForB.srcNdMatrixStride = 0; // 相邻ND矩阵起始地址之间的偏移，单位为元素个数
            mm1Nd2NzParamsForB.dstNzMatrixStride = 0; // 两个NZ矩阵，起始地址之间的偏移，单位为元素个数

            uint32_t ndNumFinish = copyStartRowCnt / blockK;
            DataCopy(bL1Tensor[ndNumFinish * blockK * headDimAlign + copyStartRowCntInABlockK * (
                // ↑ 换行续接 ↓
                32 / sizeof(KV_T))], valueGm[valueGmBaseOffset],
                    mm1Nd2NzParamsForB);
        }
    }

    if (midNdNum != 0) {
        if constexpr (KV_LAYOUT_T == LAYOUT::NZ) {
            DataCopyParams intriParams;
            intriParams.blockLen = blockK;
            intriParams.blockCount = headDim / blockElementCnt;
            intriParams.dstStride = 0;
            intriParams.srcStride = kvCacheBlockSize - blockK;

            int32_t ndNumFinish = (copyStartRowCnt + nHead) / blockK;
            for (uint32_t i = 0; i < midNdNum; i++) {
                DataCopy(bL1Tensor[(ndNumFinish + i) * blockK * headDimAlign],
                    // ↑ 换行续接 ↓
                    valueGm[valueGmBaseOffset + nHead * step + i * blockK * step], intriParams);
            }
        } else {
            int32_t ndNumFinish = (copyStartRowCnt + nHead) / blockK;
            if ((KV_LAYOUT_T == LAYOUT::BSH || KV_LAYOUT_T == LAYOUT::BSND)
                // ↑ 换行续接 ↓
                && (blockK * step > 65535)) {
                mm1Nd2NzParamsForB.ndNum = 1;
                mm1Nd2NzParamsForB.nValue = blockK; // 单个ND矩阵的行数，单位为元素个数
                mm1Nd2NzParamsForB.srcNdMatrixStride = 0; // 相邻ND矩阵起始地址之间的偏移，单位为元素个数
                mm1Nd2NzParamsForB.dstNzMatrixStride = 0; // 两个NZ矩阵，起始地址之间的偏移，单位为元素个数

                for (uint32_t i = 0; i < midNdNum; i++) {
                    DataCopy(bL1Tensor[(ndNumFinish + i) * blockK * headDimAlign],
                        // ↑ 换行续接 ↓
                        valueGm[valueGmBaseOffset + nHead * step + i * blockK * step], mm1Nd2NzParamsForB);
                }
            } else {
                mm1Nd2NzParamsForB.ndNum = midNdNum;
                mm1Nd2NzParamsForB.nValue = blockK; // 单个ND矩阵的行数，单位为元素个数
                mm1Nd2NzParamsForB.srcNdMatrixStride = blockK * step; //
                    // ↑ 换行续接 ↓
                    相邻ND矩阵起始地址之间的偏移，单位为元素个数
                mm1Nd2NzParamsForB.dstNzMatrixStride = blockK * headDimAlign;
                    // ↑ 换行续接 ↓
                    // 两个NZ矩阵，起始地址之间的偏移，单位为元素个数

                DataCopy(bL1Tensor[ndNumFinish * blockK * headDimAlign],
                    // ↑ 换行续接 ↓
                    valueGm[valueGmBaseOffset + nHead * step], mm1Nd2NzParamsForB);
            }
        }
    }

    if (nTail != 0) {
        if constexpr (KV_LAYOUT_T == LAYOUT::NZ) {
            DataCopyParams intriParams;
            intriParams.blockLen = nTail;
            intriParams.blockCount = headDim / blockElementCnt;
            intriParams.dstStride = blockK - nTail;
            intriParams.srcStride = kvCacheBlockSize - nTail;

            int32_t ndNumFinish = (copyStartRowCnt + nHead) / blockK + midNdNum;
            DataCopy(bL1Tensor[ndNumFinish * blockK * headDimAlign + copyStartRowCntInBaseNTail * (
                // ↑ 换行续接 ↓
                32 / sizeof(KV_T))], valueGm[valueGmBaseOffset + (nHead + midNdNum * blockK) * step], intriParam
s);
        } else {
            mm1Nd2NzParamsForB.ndNum = 1;
            mm1Nd2NzParamsForB.nValue = nTail; // 单个ND矩阵的行数，单位为元素个数
```

```
            mm1Nd2NzParamsForB.srcNdMatrixStride = 0; // 相邻ND矩阵起始地址之间的偏移，单位为元素个数
            mm1Nd2NzParamsForB.dstNzMatrixStride = 0; // 两个NZ矩阵，起始地址之间的偏移，单位为元素个数

            int32_t ndNumFinish = (copyStartRowCnt + nHead) / blockK + midNdNum;
            DataCopy(bL1Tensor[ndNumFinish * blockK * headDimAlign + copyStartRowCntInBaseNTail * (
                // ↑ 换行续接 ↓
                32 / sizeof(KV_T))], valueGm[valueGmBaseOffset + (nHead + midNdNum * blockK) * step],
                    mm1Nd2NzParamsForB);
        }
    }
}
```

二十一、CalcParams函数 (行2654-2760)

功能说明: 计算当前迭代的所有参数

```
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::CalcParams(
    // ↑ 换行续接 ↓
    uint32_t loop, ExtraInfo& info, TaskContext &task) {
    info.loop = loop;
    info.bIdx = task.bidx;
    info.s1Idx = task.s1idx;
    info.s2Idx = task.s2idx;
    info.n2Idx = task.nidx;
    info.curSInnerLoopTimes = task.s2loops;
    info.s1Size = task.s1Size;
    info.s2Size = task.s2Size;
    info.gIdx = task.gidx;
    info.gSize = (task.gidx == gOuter - 1) ? gSizeTail : gSizeSub;
    info.actS1Size = task.actS1Size;
    info.mSize = info.gSize * info.s1Size;
    info.actSeqLenQ = task.actSeqLenQ;

    if ASCEND_IS_AIV {
        info.mSizeV = (info.mSize + 1) / 2;
        info.mSizeVStart = 0;
        if (tmpBlockIdx % 2 == 1) {
            info.mSizeVStart = info.mSizeV;
            info.mSizeV = info.mSize - info.mSizeV;
        }
    }

    info.isFirstSInnerLoop = (info.s2Idx == 0);
    if (info.isFirstSInnerLoop) {
        bn2IdxInCurCore++;
    }
    info.bn2IdxInCurCore = bn2IdxInCurCore - 1;
    if (info.isFirstSInnerLoop) {
        if constexpr (LAYOUT_T == LAYOUT::BNSD) {
            // B,N2,G,1,D
            tensorACoreOffset = info.bIdx * qHeadNum * qSeqSize * headDim
                // ↑ 换行续接 ↓
                + info.n2Idx * gSize * qSeqSize * headDim +
                            info.gIdx * gSizeSub * qSeqSize * headDim;
            // B,N2,S2,D
            tensorBCoreOffset = info.bIdx * kvHeadNum * kvSeqSize * headDim
                // ↑ 换行续接 ↓
                + info.n2Idx * kvSeqSize * headDim;
            if (!batchContinuous) {
                uint64_t seqSize = SeqLenFromTensorList(bIdx);
                tensorBCoreOffset = info.n2Idx * seqSize * headDim;
            }
            tensorBCoreOffset += kvPaddingBeginOffset * headDim;
            if constexpr (FLASH_DECODE) {
                tensorBCoreOffset += s2IdxFD * sInnerLoopSize * headDim;
            }
        } else {
            // B,1,N2,G,D
            tensorACoreOffset = GetQueryOffSetBySIdx(info.bIdx, info.s1Idx
                // ↑ 换行续接 ↓
                * s1SizeSub, info.n2Idx, 0);
            // B,S2,N2,D
            tensorBCoreOffset = info.bIdx * kvSeqSize * kvHeadNum * headDim
                // ↑ 换行续接 ↓
```

```
            + info.n2Idx * headDim;
        if (!batchContinuous) {
            tensorBCoreOffset = info.n2Idx * headDim;
        }
        tensorBCoreOffset += kvPaddingBeginOffset * kvHeadNum * headDim;
        if constexpr (FLASH_DECODE) {
            tensorBCoreOffset += s2IdxFD * sInnerLoopSize * kvHeadNum * headDim;
        }
    }
}
info.tensorAOffset = tensorACoreOffset;
if constexpr (LAYOUT_T == LAYOUT::BNSD) {
    info.tensorBOffset = tensorBCoreOffset + info.s2Idx * singleProcessSInnerSize
        // ↑ 换行续接 ↓
        * headDim;
} else {
    info.tensorBOffset = tensorBCoreOffset + info.s2Idx * singleProcessSInnerSize
        // ↑ 换行续接 ↓
        * kvHeadNum * headDim;
}
info.attenOutOffset = tensorACoreOffset;
info.actualSingleProcessSInnerSize = singleProcessSInnerSize;
if (info.s2Idx == info.curSInnerLoopTimes - 1) {
    info.actualSingleProcessSInnerSize = task.s2SizeTail;
}
if constexpr (KVINT4) {
    info.actualSingleProcessSInnerSizeAlign = Align((uint32_t)info.actualSingleProcessSInnerSize,
        // ↑ 换行续接 ↓
        (uint32_t)64L);
} else {
    info.actualSingleProcessSInnerSizeAlign = Align((uint32_t)info.actualSingleProcessSInnerSize,
        // ↑ 换行续接 ↓
        (uint32_t)BYTE_BLOCK);
}

if (antiquantPerTensorFlag == 1) {
    info.antiqKeyParamOffset = 0;
} else if (antiquantPerHeadFlag) {
    info.antiqKeyParamOffset = info.n2Idx;
} else {
    info.antiqKeyParamOffset = info.n2Idx * headDim;
}

if (antiquantPerTensorFlag == 1) {
    info.antiqValueParamOffset = 0;
} else if (antiquantPerHeadFlag) {
    info.antiqValueParamOffset = info.n2Idx;
} else {
    info.antiqValueParamOffset = info.n2Idx * headDim;
}
uint64_t sInnerOffsetDataSize = info.s2Idx * singleProcessSInnerSize;
if constexpr (FLASH_DECODE) {
    sInnerOffsetDataSize += s2IdxFD * sInnerLoopSize;
}

info.antiqKeyParamOffsetPerToken = info.bIdx * antiqSeqSize + sInnerOffsetDataSize + kvPaddingBeginOffse
t;
    // ↑ 换行续接 ↓
    // info.n2Idx * headDim;
info.antiqValueParamOffsetPerToken = info.bIdx * antiqSeqSize + sInnerOffsetDataSize
    // ↑ 换行续接 ↓
    + kvPaddingBeginOffset;
if (antiquantPerHeadFlag) {
    info.antiqKeyParamOffsetPerToken = info.bIdx * antiqSeqSize * kvHeadNum + info.n2Idx * antiqSeqSize +
sInnerOffsetDataSize + kvPaddingBeginOffset;
        // ↑ 换行续接 ↓
        // info.n2Idx * headDim;
    info.antiqValueParamOffsetPerToken = info.bIdx * antiqSeqSize *
        // ↑ 换行续接 ↓
        kvHeadNum + info.n2Idx * antiqSeqSize + sInnerOffsetDataSize + kvPaddingBeginOffset;
}
// out quant
info.perChannelQuantOffset = info.n2Idx * headDim * gSize;
attenMaskCoreOffset = info.bIdx * attenMaskSize + kvPaddingBeginOffset;
info.attenMaskOffset = attenMaskCoreOffset + sInnerOffsetDataSize;
info.s2BatchOffset = s2BatchBaseOffset + sInnerOffsetDataSize;
}
```